

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
«Гродненский государственный университет имени Янки Купалы»

Факультет математики и информатики
Кафедра
системного программирования и компьютерной безопасности

КИСЕЛЕВСКИЙ ДАНИИЛ АНДРЕЕВИЧ

Разработка чат-бота для поиска вакансий

Курсовая работа
студента 3 курса специальности
6-05-0611-03 «Искусственный интеллект»

Научный руководитель
Вувуникян Юрий Микиртычевич,
профессор кафедры
фундаментальной и прикладной
математики,
доктор физико-математических
наук,
профессор

Гродно 2026

РЕЗЮМЕ

Курсовая работа: 81 с., 14 рис., 2 табл., 2 прил., 17 источников.

ЧАТ-БОТ, СЕМАНТИЧЕСКИЙ ПОИСК, БОЛЬШАЯ ЯЗЫКОВАЯ МОДЕЛЬ, ВЕКТОРНЫЕ ЭМБЕДДИНГИ, SENTENCE-BERT, RAG, FASTAPI, STREAMLIT, PGVECTOR, REST API.

Объект исследования: программные комплексы интеллектуального поиска вакансий с диалоговым интерфейсом, ориентированные на соискателей Республики Беларусь.

Предмет исследования: архитектура чат-бота, методы обработки естественного языка и интеграция большой языковой модели с векторной базой данных в задаче семантического поиска вакансий.

Цель работы: разработать чат-бот для семантического поиска вакансий, реализующий клиент-серверную архитектуру и интеграцию с большой языковой моделью.

Методы исследования: системный анализ предметной области, применение предобученных моделей машинного обучения и большой языковой модели, векторные методы информационного поиска, экспериментальное функциональное и количественное тестирование.

Исследования и разработки: разработан чат-бот с трёхуровневой архитектурой; реализованы сбор, векторизация вакансий, Backend с REST API, LLM-извлечение структурированных фильтров, адаптивная релаксация фильтров и доменное ранжирование, пользовательский веб-интерфейс.

Элементы научной новизны: предложена архитектура диалоговой системы поиска вакансий, объединяющая большую языковую модель для извлечения структурированных фильтров и bi-encoder для семантического поиска по векторной БД.

Область возможного практического применения: сервисы карьерных центров ВУЗов, агрегаторы вакансий, HR-платформы, корпоративные системы подбора персонала.

Автор работы подтверждает, что приведённый в курсовой работе расчётно-аналитический материал правильно и объективно отражает состояние исследуемого процесса, а все заимствованные из литературных и других источников теоретические, методологические и методические положения и концепции сопровождаются ссылками на их авторов.

SUMMARY

Course project: 81 p., 14 fig., 2 tabl., 2 app., 17 ref.

CHATBOT, SEMANTIC SEARCH, LARGE LANGUAGE MODEL, VECTOR EMBEDDINGS, SENTENCE-BERT, RAG, FASTAPI, STREAMLIT, PGVECTOR, REST API.

The object of the research: software systems for intelligent vacancy search with a dialog interface, focused on job seekers in the Republic of Belarus.

The subject of the research: chatbot architecture, natural language processing methods, and integration of a large language model with a vector database for semantic vacancy search.

The research objective: to develop a chatbot for semantic vacancy search that implements a client-server architecture and integration with a large language model.

The methods of research: system analysis of the subject domain, application of pretrained machine learning models and a large language model, vector information retrieval methods, and experimental functional and quantitative testing.

Researches and developments: a chatbot with a three-tier architecture has been developed; vacancy collection, vacancy vectorization, a backend with REST API, LLM-based structured filter extraction, adaptive filter relaxation, domain reranking, and a user web interface have been implemented.

The elements of scientific novelty: an architecture of a dialog vacancy search system is proposed, combining a large language model for structured filter extraction and a bi-encoder for semantic search over a vector database.

The field of possible practical use: services of university career centers, vacancy aggregators, HR platforms, and corporate personnel recruitment systems.

The author of work confirms that the design and analytical materials of course project reflect the state of the process under research correctly and impartially, and all theoretical, methodological and methodical statements and conceptions borrowed from the literature and other sources are accompanied by references to their authors.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ	5
ВВЕДЕНИЕ	7
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПОСТРОЕНИЯ ЧАТ-БОТОВ И СЕМАНТИЧЕСКОГО ПОИСКА	9
1.1 Чат-боты как интерфейс взаимодействия в современных интеллектуальных системах	9
1.2 Технологии семантического поиска и векторного представления текста	9
1.3 Архитектура Retrieval-Augmented Generation в задачах поиска вакансий.	10
1.4 Метрики оценки эффективности диалоговых поисковых систем	10
1.5 Выводы по главе 1	11
2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ, ПОСТАНОВКА ЗАДАЧИ И ПРОЕКТИРОВАНИЕ ЧАТ-БОТА	12
2.1 Специфика поиска работы	12
2.2 Сравнительный анализ диалоговых систем и существующих HR- платформ	12
2.3 Постановка задачи и формирование технических требований к чат-боту	14
2.4 Обоснование выбора стека технологий и архитектуры	14
2.5 Выводы по главе 2	15
3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И АРХИТЕКТУРА ЧАТ-БОТА	16
3.1 Трёхуровневая архитектура программного комплекса	16
3.2 Модель хранения данных: схема базы данных и векторное поле	17
3.3 Разработка Backend API на FastAPI	19
3.4 Логика обработки запроса: извлечение фильтров большой языковой моделью	22
3.5 Алгоритмы устойчивого поиска: релаксация фильтров и доменное ранжирование	24
3.6 Разработка пользовательского веб-интерфейса на Streamlit	27
3.7 Сбор и подготовка данных: парсинг и векторизация вакансий	29

3.8 Выводы по главе 3	30
4 ЭКСПЕРИМЕНТАЛЬНАЯ ПРОВЕРКА РАБОТОСПОСОБНОСТИ ЧАТ-БОТА.....	32
4.1 Постановка эксперимента и набор бенчмарков.....	32
4.2 Функциональное тестирование чат-бота в пользовательских сценариях..	33
4.3 Количественная оценка качества поиска.....	37
4.4 Анализ устойчивости системы и обработка ошибочных запросов	40
4.5 Визуализация процесса обработки запроса	41
4.6 Выводы по главе 4	43
5 НАПРАВЛЕНИЯ ДАЛЬНЕЙШЕГО РАЗВИТИЯ ПРОЕКТА	45
5.1 Масштабирование и продуктивизация	45
5.2 Интеграция с популярными мессенджерами	46
5.3 Реализация персонализации	46
5.4 Внедрение генеративного слоя для резюмирования вакансий.....	47
5.5 Усиление слоя извлечения кандидатов (исследовательское направление)	47
5.6 Выводы по главе 5	48
ЗАКЛЮЧЕНИЕ	49
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	51
ПРИЛОЖЕНИЕ А	53
ПРИЛОЖЕНИЕ Б	77

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

ANN	Approximate Nearest Neighbors (приближённый поиск ближайших соседей)	NLP	Natural Language Processing (обработка естественного языка)
API	Application Programming Interface (программный интерфейс)	ORM	Object-Relational Mapping
BERT	Bidirectional Encoder Representations from Transformers	RAG	Retrieval-Augmented Generation
CRUD	Create, Read, Update, Delete	REST	Representational State Transfer
DTO	Data Transfer Object (объект передачи данных)	SBERT	Sentence-BERT (Reimers, Gurevych, 2019)
HNSW	Hierarchical Navigable Small World (индекс ANN)	SPA	Single Page Application
HTTP	HyperText Transfer Protocol	SQL	Structured Query Language
IR	Information Retrieval (информационный поиск)	UI	User Interface (пользовательский интерфейс)
JSON	JavaScript Object Notation	URL	Uniform Resource Locator
LLM	Large Language Model (большая языковая модель)	UUID	Universally Unique Identifier
		ВУЗ	Высшее учебное заведение
		СУБД	Система управления базами данных

ВВЕДЕНИЕ

В условиях цифровой трансформации рынка труда все более заметным становится переход от классических форм поиска вакансий к диалоговым интерфейсам. Пользователь ожидает взаимодействия в естественной форме, когда запрос можно сформулировать свободно, без строгого знания терминологии и структуры фильтров. Традиционные платформы поиска вакансий в основном используют лексический поиск и жесткую фильтрацию, из-за чего близкие по смыслу формулировки нередко дают различные результаты. Для большинства соискателей эта проблема особенно актуальна, поскольку поиск первой работы часто сопровождается неопределенностью в выборе должности и формулировке запроса. Разработка чат-бота для семантического поиска вакансий позволяет повысить доступность и качество первичного трудоустройства за счет понимания смысла запроса, а не только совпадения отдельных слов.

Разрабатываемый чат-бот ориентирован на прикладное использование в условиях Республики Беларусь и может применяться как сервис карьерных центров высших учебных заведений, как прототип для HR-платформ, а также как учебный стенд при изучении дисциплин по веб-разработке, базам данных и искусственному интеллекту. Реализация на основе клиент-серверной архитектуры обеспечивает возможность последующего масштабирования, расширения каналов взаимодействия и интеграции в существующие цифровые экосистемы.

Теоретические основы работы базируются на современных исследованиях в области обработки естественного языка и информационного поиска. К ключевым работам относятся исследования А. Vaswani и соавторов по архитектуре Transformer, J. Devlin и соавторов по модели BERT, N. Reimers и I. Gurevych по Sentence-BERT, P. Lewis и соавторов по подходу Retrieval-Augmented Generation, а также W. Wang и соавторов по модели MiniLM. Среди промышленных решений необходимо выделить ChatGPT, Claude, Gemini и крупные платформы поиска вакансий. При этом специализированные решения, адаптированные под локальные особенности рынка труда Республики Беларусь и сочетающие диалоговый ввод с семантическим поиском по векторной базе вакансий, представлены ограниченно, что подтверждает актуальность выбранного направления разработки.

Выбор темы обусловлен сочетанием технологической и социальной значимости задачи. С одной стороны, развитие больших языковых моделей, векторных представлений текста и инструментов быстрой разработки веб-

сервисов делает возможным создание интеллектуального чат-бота в рамках курсового проекта. С другой стороны, существует практический запрос на удобные цифровые инструменты поиска работы для начинающих специалистов. Продуктовая направленность темы позволяет получить не только теоретически обоснованное, но и функционально завершённое программное решение.

Цель работы – разработать чат-бот для семантического поиска вакансий, реализующий клиент-серверную архитектуру и интеграцию с большой языковой моделью, адаптированный к рынку труда Республики Беларусь.

Задачи исследования:

1. Изучить теоретические основы построения чат-ботов, методы обработки естественного языка, семантического поиска и архитектуры Retrieval-Augmented Generation.
2. Проанализировать предметную область рынка труда Республики Беларусь, существующие диалоговые системы и сформировать требования к чат-боту.
3. Обосновать выбор технологического стека и архитектурного решения.
4. Спроектировать и реализовать программный комплекс чат-бота, включая модель данных, серверную часть, логику обработки запроса и пользовательский интерфейс.
5. Выполнить функциональное и количественное тестирование разработанного решения.
6. Предложить направления дальнейшего развития проекта.

Объект исследования являются программные комплексы интеллектуального поиска вакансий с диалоговым интерфейсом.

Предмет исследования составляют архитектура чат-бота, методы обработки естественного языка и интеграция векторной базы данных в задаче семантического поиска вакансий.

Использованы методы системного анализа предметной области, программной инженерии, информационного поиска, применения предобученных языковых моделей и экспериментальное тестирование программного продукта.

Структура работы. Курсовая работа состоит из введения, пяти глав, заключения, списка использованных источников и приложений.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПОСТРОЕНИЯ ЧАТ-БОТОВ И СЕМАНТИЧЕСКОГО ПОИСКА

В данной главе рассматриваются теоретические положения, формирующие основу разработки чат-бота для семантического поиска вакансий. Анализируются подходы к построению диалоговых систем, технологии векторного информационного поиска, принципы архитектуры Retrieval-Augmented Generation и показатели оценки качества таких систем.

1.1 Чат-боты как интерфейс взаимодействия в современных интеллектуальных системах

Чат-бот представляет собой программный агент, обеспечивающий взаимодействие пользователя с информационной системой на естественном языке. В прикладных задачах выделяют сценарные системы, системы извлечения готового ответа и генеративные системы, использующие большие языковые модели. Для задачи поиска вакансий наиболее устойчивым является подход, в котором чат-бот ориентирован на конкретную предметную область и формирует ответ на основе структурированного поиска по базе данных [3, 4].

В контексте данной работы чат-бот рассматривается как task-oriented система, ориентированная на достижение практического результата для пользователя, а именно подбор релевантных вакансий по свободному текстовому запросу. Такой подход снижает порог входа для соискателей, которые часто формулируют запрос не профессиональными терминами, а описанием желаемой деятельности.

1.2 Технологии семантического поиска и векторного представления текста

Семантический поиск отличается от лексического тем, что учитывает смысл запроса и документа, а не только совпадение ключевых слов. Исторически развитие средств поиска прошло путь от TF-IDF и BM25 к распределенным векторным представлениям и далее к контекстным трансформерным моделям. Современные bi-encoder модели семейства

Sentence-BERT позволяют эффективно получать эмбединги как для запросов, так и для вакансий, обеспечивая сравнение по косинусной близости [3].

Для многоязычных и смешанных русско-английских описаний вакансий применение компактных multilingual моделей типа MiniLM является обоснованным компромиссом между качеством и скоростью. Хранение эмбедингов в векторном поле базы данных и поиск ближайших соседей по косинусному расстоянию позволяют реализовать масштабируемый и воспроизводимый контур семантического подбора [5, 14].

1.3 Архитектура Retrieval-Augmented Generation в задачах поиска вакансий

Архитектура Retrieval-Augmented Generation предполагает совместную работу модуля извлечения релевантных документов и языковой модели. В классическом варианте retriever формирует контекст, а generator синтезирует итоговый ответ пользователя на основе найденных источников. Такой подход снижает вероятность галлюцинаций и позволяет опираться на актуальные внешние данные [4].

Важно отметить, что в рамках данной работы реализуется модифицированный подход: роль LLM сосредоточена на этапе анализа запроса и извлечения семантических фильтров, в то время как итоговая выдача формируется из верифицированных данных БД без этапа синтеза нового текста. Такое решение позволяет избежать проблемы "галлюцинаций" моделей и гарантирует достоверность предоставляемой соискателю информации, что является критически важным для сервисов поиска работы. Таким образом, систему можно классифицировать как интеллектуальный семантический поисковик с перспективой перехода к полноценному RAG [4].

1.4 Метрики оценки эффективности диалоговых поисковых систем

Оценка эффективности чат-бота для поиска вакансий должна включать как поисковые, так и системные показатели. К поисковым метрикам относятся precision, recall, hit@k, MRR и nDCG. Для прикладной проверки в рамках курсовой работы целесообразно использовать hit@1 и hit@5, так как

они напрямую отражают вероятность того, что пользователь увидит релевантный вариант в первых позициях выдачи.

Системная оценка дополняется измерением времени отклика API, корректности обработки неструктурированного запроса и устойчивости к ошибкам внешних сервисов. Комплексное применение указанных метрик обеспечивает объективную проверку работоспособности и практической значимости разработанного чат-бота для рынка труда Республики Беларусь.

1.5 Выводы по главе 1

1. Установлено, что для задачи поиска вакансий наиболее эффективно применение предметно-ориентированного чат-бота, объединяющего диалоговый интерфейс и структурированный поиск.
2. Показано, что семантический поиск на базе векторных представлений обеспечивает более релевантные результаты по сравнению с чисто лексическими методами.
3. Обосновано использование архитектуры Retrieval-Augmented Generation как основы для объединения языковой модели и внешней базы вакансий [4].
4. Определены основные метрики оценки качества системы, включая hit@1 , hit@5 и показатели времени отклика API, применимые в последующих главах.

2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ, ПОСТАНОВКА ЗАДАЧИ И ПРОЕКТИРОВАНИЕ ЧАТ-БОТА

Во второй главе выполняется переход от общетеоретических положений к прикладному анализу предметной области и формальной постановке задачи разработки чат-бота. Рассматриваются особенности рынка труда Республики Беларусь, анализируются существующие решения, формулируются функциональные и нефункциональные требования к системе, а также обосновывается выбор технологического стека и архитектурных подходов.

2.1 Специфика поиска работы

Поиск первой работы характеризуется высокой неопределенностью требований и терминологии. Пользователь, как правило, формулирует запрос не в профессиональной нотации вакансий, а в описательной форме, указывая желаемый характер деятельности, условия графика и уровень стартовой нагрузки. В результате классические лексические поисковые механизмы, опирающиеся на точные совпадения ключевых слов, часто возвращают неполные или нерелевантные результаты.

Дополнительной особенностью является мультязычный характер вакансий: русскоязычные описания обычно содержат англоязычные вкрапления технических терминов и названий должностей. Для корректной интерпретации таких запросов необходимо сочетание семантического поиска по векторным представлениям и интеллектуального извлечения структурированных параметров из свободного текста. Это формирует обоснование выбора архитектуры чат-бота как инструмента, ориентированного на естественный язык пользователя, а не на строгий набор формальных фильтров.

2.2 Сравнительный анализ диалоговых систем и существующих HR-платформ

На современном этапе можно выделить две группы решений, частично пересекающихся с задачей исследования: универсальные диалоговые

системы и специализированные HR-платформы. Универсальные системы, такие как ChatGPT, Claude и Gemini, демонстрируют высокое качество языкового взаимодействия, но не обеспечивают встроенный доступ к актуальной локальной базе вакансий. Их ответы нередко носят консультационный характер и не заменяют полноценного механизма подбора вакансий по структурированным ограничениям.

HR-платформы, включая rabota.by, HeadHunter и LinkedIn, напротив, предоставляют реальные данные о вакансиях, но в большинстве случаев реализуют поиск через комбинацию фильтров и лексических совпадений. Такой подход ограниченно поддерживает синонимические формулировки и плохо работает при нечетком пользовательском запросе. Сравнительный анализ показывает, что большинства пользователей требуется гибридное решение, объединяющее достоинства чат-интерфейса и семантического retrieval-подхода на локальном корпусе вакансий.

Таблица 2.1 – Сравнительный анализ универсальных диалоговых систем, HR-платформ и разрабатываемого чат-бота

Критерий	Универсальные диалоговые системы (ChatGPT, Claude, Gemini)	Специализированные HR-платформы (rabota.by, HeadHunter, LinkedIn)	Разрабатываемый чат-бот
1	Работа в пункте выдачи заказов Wildberries или Ozon, без опыта	Продавец-кассир в пункт выдачи заказов, город Гродно	да
3	Бариста или работа в кофейне, без опыта	Бариста (ИП Христинченко Татьяна Петровна)	да
6	Работа в логистике: логист, экспедитор, график 5 на 2	Менеджер по логистике (ООО К 2 Технологии)	да
8	Удалённая вакансия support и help desk	Support Specialist (Help Desk Analyst), GP Solutions	да

12	Стартовая роль в подборе персонала, IT-сфера, junior	Junior Sales Manager (ООО Сенлайн)	нет
----	--	------------------------------------	-----

Окончание таблицы 2.1

Примечание – Источник: собственная разработка.

2.3 Постановка задачи и формирование технических требований к чат-боту

Целью проектирования является создание программного комплекса, позволяющего пользователю формулировать запрос в естественной форме и получать релевантную подборку вакансий с учетом профессионального контекста, режима занятости и иных ограничений. Для достижения цели формализуются ключевые пользовательские сценарии: ввод свободного запроса, получение ранжированного списка вакансий, просмотр карточек с основными параметрами и переход к детальному описанию позиции.

К функциональным требованиям относятся автоматическое наполнение базы вакансий, векторизация текстов на основе мультязычной модели, извлечение фильтров через большую языковую модель, релаксация избыточно строгих ограничений и выдача топ-результатов через веб-интерфейс и REST API. Нефункциональные требования включают устойчивость к ошибкам внешнего LLM-сервиса, валидацию входных данных, мультязычную обработку запросов и обеспечение времени отклика API на уровне не более двух секунд в типовых сценариях использования.

Ожидаемые результаты разработки включают: получение работоспособного end-to-end решения, формирование базы знаний вакансий с векторными представлениями, достижение воспроизводимых метрик качества hit@1 и hit@5 на эталонном наборе запросов, а также подтверждение корректной работы пользовательского диалога и сервера обработки запросов в рамках функционального тестирования.

2.4 Обоснование выбора стека технологий и архитектуры

В качестве языка разработки выбран Python, обеспечивающий единую технологическую среду для backend-логики, обработки данных и использования современных NLP-библиотек. Серверная часть реализуется на FastAPI, что позволяет использовать асинхронный режим обработки запросов, строгую типизацию и автоматическое формирование OpenAPI-документации. Клиентский уровень реализуется на Streamlit, предоставляющем удобный интерфейс для построения диалоговой формы взаимодействия и визуализации результатов поиска.

Для хранения данных используется PostgreSQL с расширением pgvector, которое обеспечивает поддержку векторных типов и операций косинусной близости. Модель эмбедингов формируется на базе Sentence-BERT, что позволяет сравнивать запрос и вакансии по смысловому сходству. В качестве LLM-компонента применяется внешний сервис Groq, используемый для извлечения структурированных фильтров из пользовательского текста. Такое сочетание технологий формирует трехуровневую архитектуру клиент-сервер-база данных с четким разделением ответственности между компонентами.

2.5 Выводы по главе 2

1. Установлено, что специфика рынка труда Республики Беларусь и пользовательских запросов требует перехода от лексического поиска к семантическому подходу.
2. Сравнительный анализ показал, что существующие универсальные чат-системы и HR-платформы не обеспечивают одновременно диалоговый ввод и устойчивый семантический подбор вакансий.
3. Сформулирована постановка задачи разработки чат-бота и зафиксированы ключевые функциональные и нефункциональные требования к программному комплексу.
4. Обоснован технологический стек и архитектура системы, позволяющие реализовать масштабируемое решение на основе FastAPI, Streamlit, PostgreSQL с pgvector, Sentence-BERT и LLM-сервиса Groq.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И АРХИТЕКТУРА ЧАТ-БОТА

Третья глава посвящена практической реализации разработанного чат-бота и раскрывает архитектурные решения, обеспечивающие устойчивую работу системы в условиях реального пользовательского взаимодействия. В отличие от аналитической части, здесь акцент сделан на программных модулях, форматах данных, взаимодействии компонентов и логике обработки запроса на всех этапах, от ввода текста пользователем до формирования выдачи релевантных вакансий.

Программный комплекс построен по клиент-серверной модели и включает пользовательский интерфейс на Streamlit, серверную часть на FastAPI, базу данных PostgreSQL с расширением pgvector и внешний сервис большой языковой модели для извлечения структурированных фильтров из естественного языка. Такое разделение обеспечивает модульность, расширяемость и возможность дальнейшей интеграции с внешними каналами, включая мессенджеры.

3.1 Трёхуровневая архитектура программного комплекса

Архитектура решения реализована в виде трёх логических уровней: клиентского, серверного и уровня хранения данных. Клиентский слой отвечает за диалоговый интерфейс и отображение результатов, серверный слой выполняет оркестрацию бизнес-логики поиска, а база данных обеспечивает хранение и быстрый доступ к структурированным и векторным представлениям вакансий.

Поток обработки пользовательского запроса организован следующим образом: пользователь формулирует запрос в свободной форме, frontend передаёт его в REST API, backend извлекает фильтры и ключевые ролевые признаки, выполняет семантический поиск по векторной базе и возвращает ранжированный список вакансий в стандартизированном формате. Такая схема позволяет изолировать ответственность каждого уровня и снизить связанность компонентов.

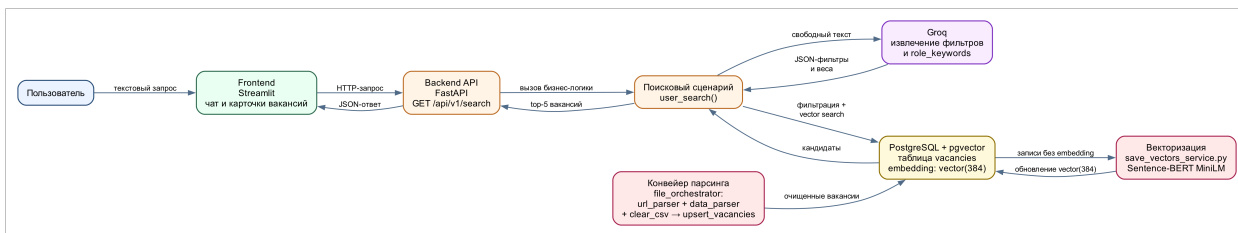


Рисунок 3.1 – Диаграмма компонентов программного комплекса и потоков данных

Примечание – Источник: собственная разработка.

3.2 Модель хранения данных: схема базы данных и векторное поле

В основе хранения данных используется таблица вакансий с набором структурированных атрибутов, описывающих наименование должности, компанию, формат занятости, требования к опыту, график работы, навыки и текстовое описание. Дополнительно для каждой записи сохраняется эмбединг фиксированной размерности, используемый для семантического сопоставления с запросом пользователя.

Выбор PostgreSQL с расширением pgvector позволяет объединить транзакционную надёжность реляционной базы и операции векторного поиска в пределах единой СУБД. ORM-модель реализована средствами SQLAlchemy, а изменения структуры базы фиксируются миграциями Alembic, что обеспечивает воспроизводимость развёртывания. Полные листинги соответствующих модулей приведены в листингах A.1 и A.2 приложения А.

vacancies	
id	SERIAL, PK
title	VARCHAR(512)
company	VARCHAR(512)
salary	VARCHAR(256)
payment_frequency	VARCHAR(128)
experience	VARCHAR(256)
employment	VARCHAR(256)
hiring_format	VARCHAR(256)
schedule	VARCHAR(256)
hours	VARCHAR(256)
work_format	VARCHAR(256)
skills	TEXT
url	VARCHAR(2048), UNIQUE
description	TEXT
embedding	vector(384)

Рисунок 3.2 – Схема таблицы vacancies в базе данных PostgreSQL

Примечание – Источник: собственная разработка.

```

from sqlalchemy import String, Text, UniqueConstraint
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from pgvector.sqlalchemy import Vector

class Base(DeclarativeBase):
    pass

class Vacancy(Base):
    __tablename__ = "vacancies"
    __table_args__ = (UniqueConstraint("url",
name="uq_vacancies_url"),)

```

```

        id: Mapped[int] = mapped_column(primary_key=True,
autoincrement=True)
        title: Mapped[str] = mapped_column(String(512),
nullable=False)
        company: Mapped[str] = mapped_column(String(512),
nullable=False)
        # ... строковые поля salary ... work_format
        skills: Mapped[str] = mapped_column(Text,
nullable=False)
        url: Mapped[str] = mapped_column(String(2048),
nullable=False)
        description: Mapped[str] = mapped_column(Text,
nullable=False)
        embedding: Mapped[list[float] | None] =
mapped_column(Vector(384), nullable=True)

```

Листинг 3.1 – Файл backend/src/features/database/models.py

Примечание – Источник: собственная разработка.

```

from alembic import op

def upgrade() -> None:
    op.execute("CREATE EXTENSION IF NOT EXISTS vector")
    op.execute(
        """
        CREATE TABLE vacancies (
            id SERIAL PRIMARY KEY,
            title VARCHAR(512) NOT NULL,
            company VARCHAR(512) NOT NULL,
            -- ... столбцы salary, payment_frequency, ...,
description (как в модели Vacancy)
            embedding vector(384),
            CONSTRAINT uq_vacancies_url UNIQUE (url)
        )
        """
    )

def downgrade() -> None:
    op.execute("DROP TABLE IF EXISTS vacancies")

```

Листинг 3.2 – Файл backend/migrations/versions/0001_create_vacancies.py

Примечание – Источник: собственная разработка.

3.3 Разработка Backend API на FastAPI

Серверная часть реализована на FastAPI как набор маршрутов, обслуживающих запросы frontend-приложения и предоставляющих

унифицированный доступ к бизнес-логике системы. Ключевым эндпоинтом является маршрут поиска вакансий, принимающий текстовый запрос пользователя и параметры ограничения выдачи, после чего формирует структурированный ответ с массивом найденных позиций.

Для входных и выходных данных используются модели валидации, что обеспечивает строгую типизацию параметров и единообразный формат ответов при штатной работе и при ошибках. Такой подход повышает надёжность интеграции между frontend и backend и упрощает сопровождение API. Полный листинг маршрутов и DTO-моделей приведён в листинге А.3 приложения А.

```
from dataclasses import asdict
from fastapi import APIRouter, HTTPException, Query
from pydantic import BaseModel, HttpUrl
# ... импорты констант и ApiErrorResponse

router = APIRouter(prefix=API_V1_PREFIX, tags=[SEARCH_TAG])
class VacancySearchResponse(BaseModel):
    id: int
    title: str
    company: str
    # ... остальные поля вакансии
    cosine_distance: float

    @router.get(SEARCH_ENDPOINT, response_model=list[VacancySearchResponse], ...)
    async def search_vacancies(
        query: str = Query(..., min_length=QUERY_MIN_LENGTH, ...),
        limit: int = Query(default=MAX_VACANCY_RESULTS, ge=1, le=MAX_VACANCY_RESULTS),
    ) -> list[VacancySearchResponse]:
        normalized_query = query.strip()
        if not normalized_query:
            raise HTTPException(status_code=HTTP_BAD_REQUEST, detail=ERROR_MESSAGE_BLANK_QUERY)
        try:
            search_result = await user_search(normalized_query, limit=limit)
        except RuntimeError as error:
            raise HTTPException(
                status_code=HTTP_INTERNAL_SERVER_ERROR,
                detail=ERROR_MESSAGE_SEARCH_UNAVAILABLE,
            ) from error
        return [
            VacancySearchResponse.model_validate(asdict(vacancy))
            for vacancy in search_result.vacancies
        ]
```

Листинг 3.3 – Файл backend/src/features/search/router.py (маршрут поиска и обработка ошибок)

Примечание – Источник: собственная разработка.

```

from dataclasses import dataclass
from features.embedding.schemas import
SimilaritySearchResult

@dataclass(frozen=True, slots=True)
class FilterRelaxStep:
    step: int
    filter_key: str
    filter_value: str
    removed_weight: float
    value_count_in_db: int
    candidates_before: int
    candidates_after: int

@dataclass(frozen=True, slots=True)
class UserSearchResult:
    user_query: str
    role_keywords: list[str]
    llm_filters: dict[str, list[dict[str, float | str]]]
    all_value_counts: dict[str, dict[str, int]]
    selected_value_counts: dict[str, dict[str, int]]
    applied_filters: dict[str, list[str]]
    dropped_filters: dict[str, list[str]]
    relax_steps: list[FilterRelaxStep]
    candidate_count: int
    vacancies: list[SimilaritySearchResult]

```

Листинг 3.4 – Файл backend/src/features/search/schemas.py (структуры результатов поиска и шага релаксации)

Примечание – Источник: собственная разработка.

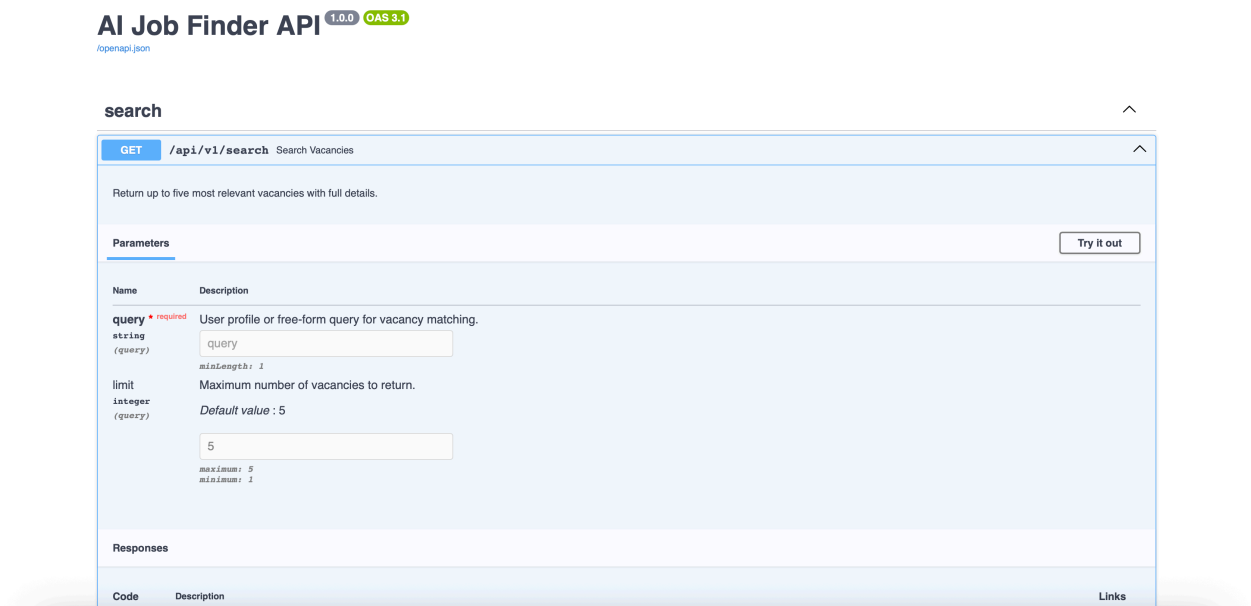


Рисунок 3.3 – Автоматически генерируемая документация OpenAPI (Swagger UI) для REST API

Примечание – Источник: собственная разработка.

3.4 Логика обработки запроса: извлечение фильтров большой языковой моделью

После получения пользовательского запроса backend передаёт текст в сервис языковой модели, который преобразует естественное описание намерения в структурированный набор фильтров по ключевым полям вакансии. В результате свободная формулировка пользователя становится пригодной для дальнейшего программного отбора и ранжирования.

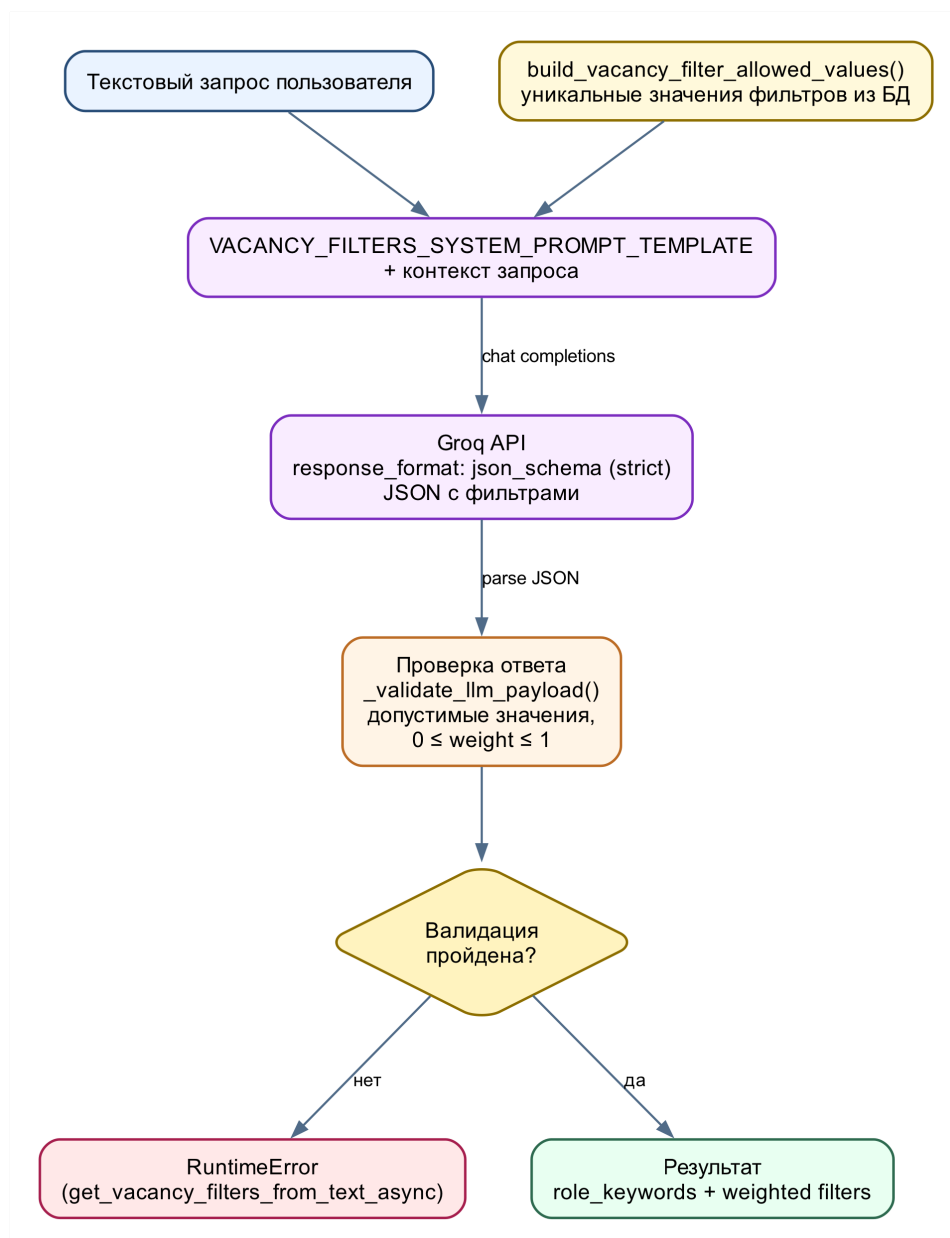


Рисунок 3.4 – Схема извлечения структурированных фильтров с помощью Groq Chat Completions и последующей валидации ответа

Примечание – Источник: собственная разработка.

На этапе постобработки выполняется проверка структуры полученного ответа, допустимых значений и полноты обязательных полей. Это необходимо для обеспечения стабильной работы дальнейших модулей и предотвращения распространения некорректных данных по пайплайну обработки. Полный листинг модуля LLM-сервиса вынесен в листинг А.4 приложения А.

```

GROQ_BASE_URL = "https://api.groq.com/openai/v1/chat/
completions"
GROQ_MODEL = "openai/gpt-oss-120b"
GROQ_TIMEOUT_SECONDS = 45

VACANCY_FILTER_KEYS = (
    "payment_frequency",
    "experience",
    "employment",
    "hiring_format",
    "schedule",
    "hours",
    "work_format",
)

VACANCY_OPTIONAL_KEYS = ("role_keywords",)

VACANCY_FILTERS_SYSTEM_PROMPT_TEMPLATE = """
Ты - узкоспециализированный ассистент, который переводит
запрос кандидата ...
в структурированные JSON-фильтры.
ПРАВИЛА:
1. Ответ - только валидный JSON (без markdown).
2. JSON содержит все поля фильтров и role_keywords.
3. Значения - только из списка допустимых (подставляется в
шаблон) .
... (полный текст промпта и маппинг синонимов - в приложении
А)
{allowed_values_json}
""".strip()

```

Листинг 3.5 – Файл backend/src/common/constants/llm.py (фрагмент: ключевые константы и шаблон системного промпта)

Примечание – Источник: собственная разработка.

3.5 Алгоритмы устойчивого поиска: релаксация фильтров и доменное ранжирование

Практика эксплуатации поисковых сценариев показывает, что чрезмерно жёсткие фильтры могут приводить к пустой выдаче даже при наличии релевантных вакансий в базе. Для устранения данной проблемы реализован механизм адаптивной релаксации: при недостаточном количестве кандидатов система последовательно ослабляет второстепенные ограничения, сохраняя приоритет наиболее значимых условий.

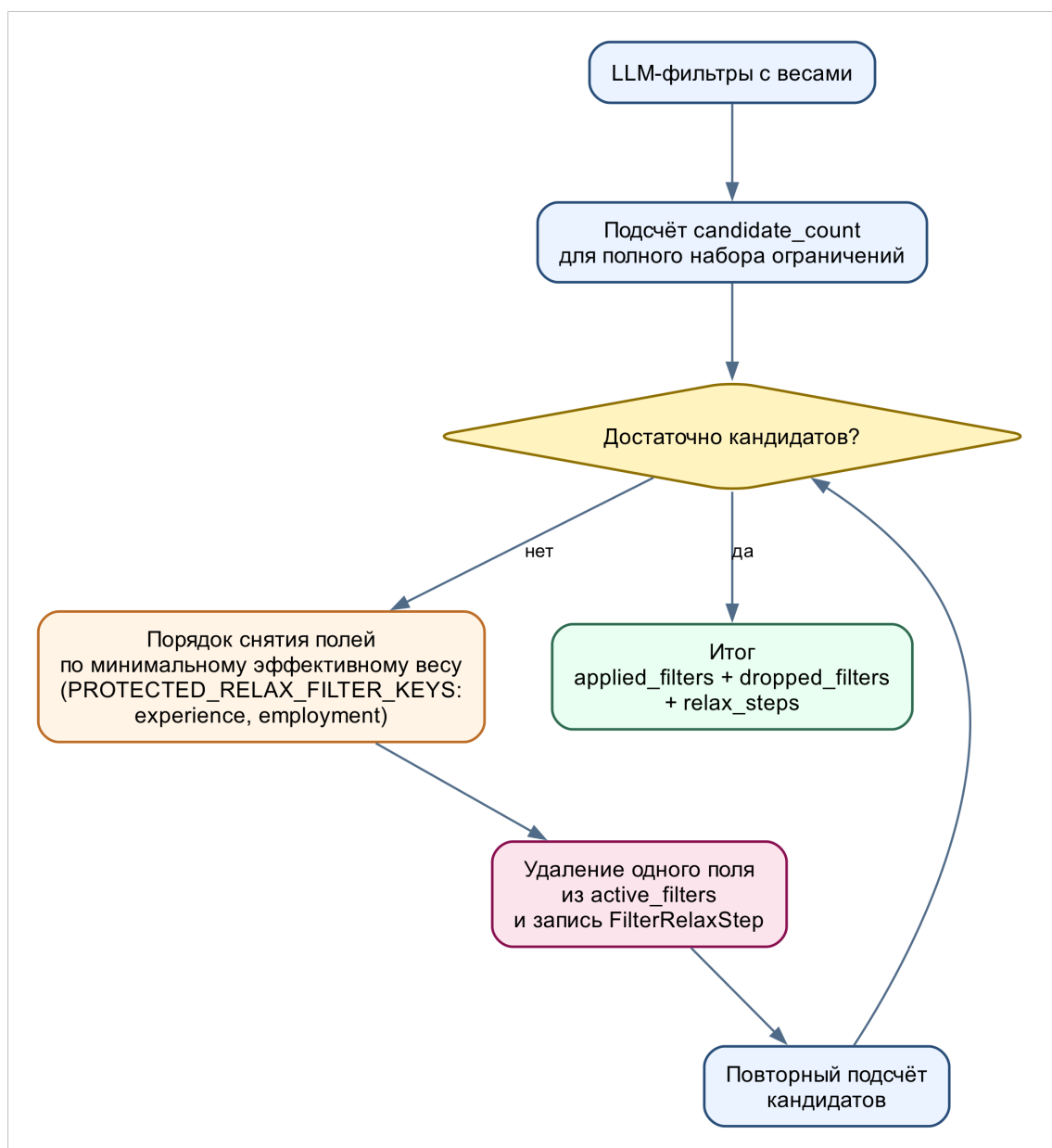


Рисунок 3.5 – Блок-схема адаптивной релаксации LLM-фильтров

Примечание – Источник: собственная разработка.

После получения кандидатов применяется доменное ранжирование, учитывающее совпадение ролевых ключевых слов и профессионального контекста запроса. Это позволяет повысить качество итогового порядка выдачи и снизить вероятность того, что формально близкие, но тематически нерелевантные вакансии окажутся в верхних позициях. Полный листинг алгоритмов релаксации и ранжирования приведён в листинге А.6 приложения А.

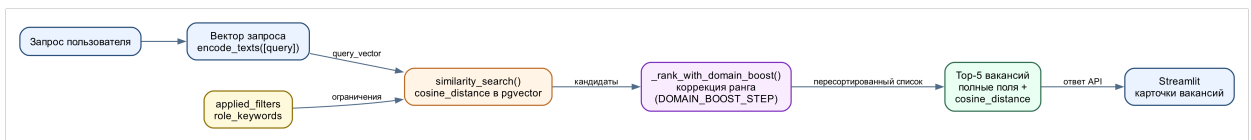


Рисунок 3.6 – Схема векторного поиска и доменного ранжирования результатов

Примечание – Источник: собственная разработка.

```

# backend/src/common/constants/search.py
DEFAULT_MIN_FILTERED_CANDIDATES = 1
DEFAULT_MAX_VALUES_PER_FILTER = 2
PROTECTED_RELAX_FILTER_KEYS = ("experience", "employment")
PROTECTED_RELAX_WEIGHT_PENALTY = 0.35
ROLE_KEYWORDS_HARD_LOCK = True
DOMAIN_BOOST_STEP = 0.03

# backend/src/features/search/service.py - фрагмент
_relax_filters
async def _relax_filters(weighted_filters, *, value_counts,
role_keywords, min_candidates):
    active_filters = _clone_non_empty(_plain_filters_from_weighted(weighted_filters))
    weighted_by_key = {k: list(v) for k, v in weighted_filters.items()}
    dropped_filters = _empty_filters()
    relax_steps: list[FilterRelaxStep] = []
    candidate_count = await _count_filtered_candidates(
        active_filters, role_keywords=role_keywords
    )
    if candidate_count >= min_candidates:
        return _with_all_keys(active_filters),
dropped_filters, relax_steps, candidate_count

    protected_keys = set(PROTECTED_RELAX_FILTER_KEYS)
    drop_order = _build_field_drop_order(weighted_by_key,
protected_keys)
    for step_index, (key, weight) in enumerate(drop_order,
start=1):
        removed_values = [value for value, _ in
weighted_by_key.get(key, [])]
        if not removed_values:
            continue
        candidates_before = candidate_count
        dropped_filters[key].extend(removed_values)
        active_filters.pop(key, None)
        candidate_count = await _count_filtered_candidates(
            active_filters, role_keywords=role_keywords
        )
        removed_total_count = sum(
            value_counts.get(key, {}).get(value, 0) for
value in removed_values
        )
        relax_steps.append(
            FilterRelaxStep(

```

```

        step=step_index,
        filter_key=key,
        filter_value=", ".join(removed_values),
        removed_weight=weight,
        value_count_in_db=removed_total_count,
        candidates_before=candidates_before,
        candidates_after=candidate_count,
    )
)
if candidate_count >= min_candidates:
    break
return _with_all_keys(active_filters), dropped_filters,
relax_steps, candidate_count

```

Листинг 3.6 – Файл backend/src/common/constants/search.py и фрагмент backend/src/features/search/service.py (релаксация фильтров)

Примечание – Источник: собственная разработка.

3.6 Разработка пользовательского веб-интерфейса на Streamlit

Пользовательский интерфейс реализован на Streamlit и организован в формате диалогового окна с сохранением истории сообщений в рамках сессии. Интерфейс поддерживает ввод запроса в естественной форме, отображение ответа ассистента и визуализацию карточек вакансий с ключевыми атрибутами для быстрого сравнения результатов.

Frontend взаимодействует с backend через HTTP-запросы и обрабатывает как успешные ответы, так и сообщения об ошибках без прерывания пользовательского сценария. Данный подход обеспечивает целостный пользовательский опыт и делает систему пригодной для демонстрации и последующего внедрения в сервисы карьерной поддержки. Полный листинг клиентских модулей вынесен в листинг А.8 приложения А.

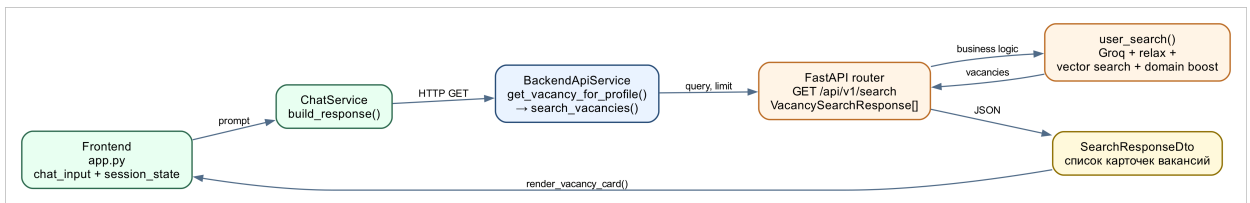


Рисунок 3.7 – Схема взаимодействия клиента Streamlit с Backend API при формировании ответа чата

Примечание – Источник: собственная разработка.

```

from dto.chat_dto import ChatMessageDto
from dto.vacancy_dto import VacancyDto
from services.backend_api_service import BackendApiService

class ChatService:
    def __init__(self) -> None:
        self.backend_service = BackendApiService()

    @staticmethod
    def _sort_vacancies_by_similarity(vacancies:
list[VacancyDto]) -> list[VacancyDto]:
        return sorted(
            vacancies,
            key=lambda v: (
                v.cosine_distance is None,
                v.cosine_distance if v.cosine_distance is
not None else float("inf"),
            ),
        )

    def build_response(self, user_prompt: str) ->
ChatMessageDto:
        analyzed =
self.backend_service.get_vacancy_for_profile(user_prompt)
        if not analyzed.vacancies:
            return ChatMessageDto(role="assistant",
content=analyzed.summary)

        sorted_vacancies =
self._sort_vacancies_by_similarity(analyzed.vacancies)
        count = len(sorted_vacancies)
        text = f"{analyzed.summary}\n\nНайдено вакансий:
**{count}**."
        return ChatMessageDto(
            role="assistant", content=text,
vacancies=sorted_vacancies
        )
  
```

Листинг 3.7 – Файл frontend/src/services/chat_service.py

Примечание – Источник: собственная разработка.

3.7 Сбор и подготовка данных: парсинг и векторизация вакансий

Отдельный функциональный контур системы отвечает за формирование актуальной базы знаний вакансий. Источником данных является специализированный портал поиска работы, откуда автоматически извлекаются карточки вакансий с сохранением значимых структурных полей и первичной очисткой текстового содержания.

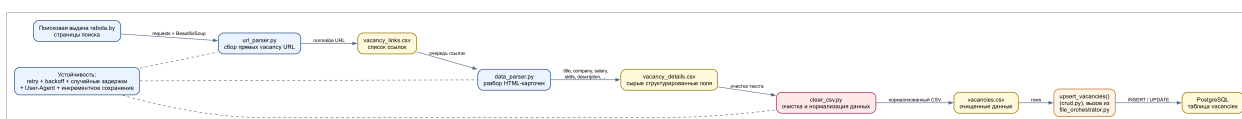


Рисунок 3.8 – Схема конвейера парсинга вакансий и загрузки данных в PostgreSQL

Примечание – Источник: собственная разработка.

На следующем этапе выполняется векторизация подготовленных текстов с использованием мультиязычной bi-encoder модели, что позволяет обеспечить корректную работу семантического поиска по русскоязычным и смешанным русско-английским формулировкам. Полученные эмбединги сохраняются в базе данных и используются в online-поиске без дополнительного вычисления в момент пользовательского запроса. Полный листинг модулей парсинга и векторизации приведён в приложении А.

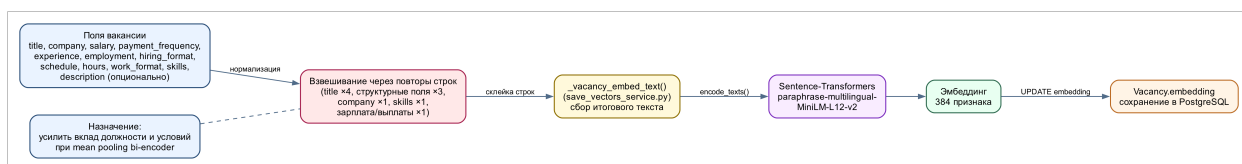


Рисунок 3.9 – Схема формирования эмбединга вакансии и сохранения вектора в базе данных

Примечание – Источник: собственная разработка.

```
# backend/src/common/constants/embedding.py
```

```

v2" EMBEDDING_MODEL_NAME = "paraphrase-multilingual-MiniLM-L12-
    EMBEDDING_DIMENSION = 384
    DEFAULT_SIMILARITY_TOP_K = 5
    VACANCY_EMBED_TITLE_REPEATS = 4
    VACANCY_EMBED_STRUCTURE_REPEATS = 3
    VACANCY_EMBED_COMPANY_REPEATS = 1
    VACANCY_EMBED_SKILLS_REPEATS = 1
    VACANCY_EMBED_INCLUDE_DESCRIPTION = True

    # save_vectors_service.py - ядро _vacancy_embed_text (без
    вспомогательных функций)
    def _vacancy_embed_text(row) -> str:
        chunks: list[str] = []
        title = _trim(row.title)
        if title:
            job_line = f"Должность: {title}"
            chunks.extend([job_line] *
VACANCY_EMBED_TITLE_REPEATS)
            chunks.extend(_repeat_lines("Компания", row.company,
VACANCY_EMBED_COMPANY_REPEATS))
            # ... для пар (label, attr) из _STRUCTURE_FIELDS -
            _repeat_lines(..., VACANCY_EMBED_STRUCTURE_REPEATS)
            chunks.extend(_repeat_lines("Зарплата", row.salary, 1))
            chunks.extend(_repeat_lines("Выплаты",
row.payment_frequency, 1))
            if not _skip_value(row.skills):
                sk = f"Навыки: {_trim(row.skills)}"
                chunks.extend([sk] * VACANCY_EMBED_SKILLS_REPEATS)
            desc = _description_for_embed(row.description)
            if desc:
                chunks.append(f"Описание: {desc}")
        return "\n\n".join(chunks)

```

Листинг 3.8 – Файлы backend/src/common/constants/embedding.py и фрагмент backend/src/features/embedding/save_vectors_service.py (взвешенный текст для эмбединга)

Примечание – Источник: собственная разработка.

3.8 Выводы по главе 3

1. В рамках третьей главы разработана и реализована архитектура программного комплекса чат-бота, обеспечивающая разделение клиентского интерфейса, серверной логики и уровня хранения данных.
2. Сформирована модель базы данных с поддержкой векторного поиска, реализован backend API с валидацией данных, внедрена логика извлечения фильтров языковой моделью, а также разработаны

алгоритмы релаксации ограничений и доменного ранжирования результатов.

3. Реализован пользовательский интерфейс диалогового взаимодействия и подготовлен контур автоматизированного наполнения базы вакансий с последующей векторизацией, что в совокупности формирует работоспособную end-to-end систему семантического поиска вакансий. Полные листинги ключевых модулей представлены в приложении А.

4 ЭКСПЕРИМЕНТАЛЬНАЯ ПРОВЕРКА РАБОТОСПОСОБНОСТИ ЧАТ-БОТА

Четвёртая глава посвящена экспериментальной проверке работоспособности разработанного чат-бота для семантического поиска вакансий. В отличие от теоретико-исследовательских работ, в которых центральное место занимает сравнение моделей, в настоящей работе акцент сделан на функциональной проверке программного продукта от лица пользователя и количественной оценке качества поисковой выдачи на эталонном наборе бенчмарк-запросов.

Программа экспериментальной проверки включает: постановку методики тестирования и описание набора эталонных запросов, функциональное тестирование четырёх типовых пользовательских сценариев со скриншотами веб-интерфейса, количественную оценку качества поиска по метрикам `hit@1` и `hit@5`, анализ устойчивости системы при обработке заведомо узких запросов и пограничных значений, а также визуализацию потока управления чат-бота в виде диаграммы активности языка UML.

4.1 Постановка эксперимента и набор бенчмарков

Целью экспериментальной части является воспроизводимая количественная и функциональная проверка того, насколько разработанный чат-бот способен возвращать релевантные вакансии по свободно сформулированному запросу пользователя. Эксперимент выполняется в том же программном контуре, что и прикладной режим: пользовательский запрос проходит этап извлечения структурированных фильтров большой языковой моделью, при необходимости активируется алгоритм релаксации фильтров, после чего выполняется векторный поиск в `pgvector` и доменное ранжирование.

В качестве эталона используется встроенный модуль автоматизированной оценки, расположенный в файле `backend/src/features/search/evaluation.py`. Модуль содержит набор из шестнадцати контрольных запросов, охватывающих типовые сценарии поиска работы соискателями. В набор включены запросы по работе в пункте выдачи заказов Wildberries и Ozon, вакансиям бариста и работе в кофейне, складским ролям (кладовщик, комплектовщик, сортировщик), логистике (логист, экспедитор, водитель), сфере информационных технологий (программист 1С, support и help desk),

маркетингу (ассистент интернет-маркетолога, digital), подбору персонала, общественному питанию (официант, повар, кондитер) и торговле (продавец, мерчендайзер).

Для каждого тестового запроса вручную задан кортеж ожидаемых ключевых слов, отражающий целевую профессиональную роль. Оценивание выполняется двумя метриками. Метрика `hit@1` принимает значение 1, если вакансия в первой позиции выдачи содержит ожидаемый профессиональный маркер в названии или в навыках, и 0 в противном случае. Метрика `hit@5` принимает значение 1, если ожидаемый маркер встречается хотя бы в одной из пяти верхних позиций выдачи. В качестве дополнительного индикатора уверенности модели фиксируется значение косинусного расстояния для `top-1` (чем ближе к нулю, тем выше семантическая близость запроса и вакансии).

Эксперимент проводится в асинхронном режиме оркестратором `run_search_evaluation`. Между обращениями к языковой модели выдерживается задержка в шестьдесят секунд для соблюдения ограничений частоты обращений к Groq Cloud Chat Completions API. Полный протокол тестирования по всем шестнадцати запросам с указанием косинусных расстояний и идентификаторов вакансий вынесен в приложение Б; в основном тексте главы приводится сокращённая выборка показательных результатов и итоговая сводка.

4.2 Функциональное тестирование чат-бота в пользовательских сценариях

Функциональное тестирование выполнено в режиме `end-to-end`: запущена локальная база данных PostgreSQL с расширением `pgvector`, применены миграции Alembic, выполнен цикл парсера портала `rabota.by` с последующей векторизацией описаний; серверная часть запущена на порту 8000, клиентская часть на основе Streamlit запущена на порту 8501. Ниже описаны четыре характерных пользовательских сценария, иллюстрирующих ключевые особенности обработки свободного запроса.

На рисунке 4.1 приведён сценарий поиска работы бариста в сеть кофеен без опыта. Пользователь формулирует запрос в свободной форме; чат-бот сообщает о числе найденных вакансий и выводит карточку с полями должности, компании, условий занятости и навыков. В показанном тесте найдены две вакансии; на первой позиции размещается вакансия «Бариста» у индивидуального предпринимателя Христиненко Татьяны Петровны с графиком «три на два» либо «два на два», форматом работы «на месте

работодателя» и формулировкой опыта «не требуется», что согласуется с запросом пользователя о начальной роли в общественном питании.

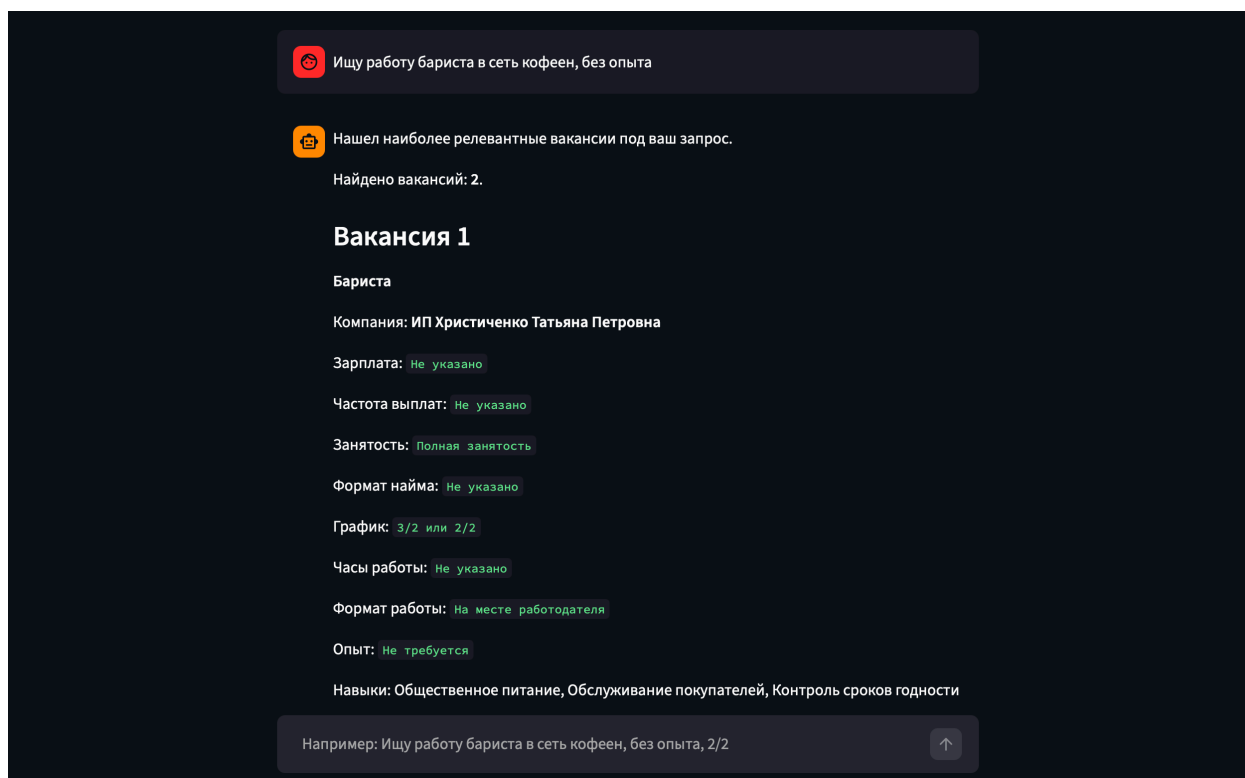


Рисунок 4.1 – Интерфейс чат-бота: поиск работы бариста в сеть кофеен без опыта

Примечание – Источник: собственная разработка.

На рисунке 4.2 представлен сценарий, в котором пользователь выражает желание работать в сфере разработки программного обеспечения или бизнес-аналитики и указывает отсутствие опыта. Чат-бот возвращает одну вакансию и отображает карточку «Бизнес-аналитик/специалист по внедрению программного обеспечения» в компании «ООО МигСофт» с полной занятостью, графиком «пять на два», восьмичасовым днём, форматом работы «на месте работодателя или удалённо» и формулировкой опыта «от одного до трёх лет». Наблюдается типичное ограничение семантического поиска: формулировка «опыта нет» в запросе не гарантирует строгое совпадение по полю опыта в карточке вакансии; тем не менее выдача остаётся в смежном IT-домене и иллюстрирует поведение чат-бота при неоднозначном соответствии жёстких атрибутов.

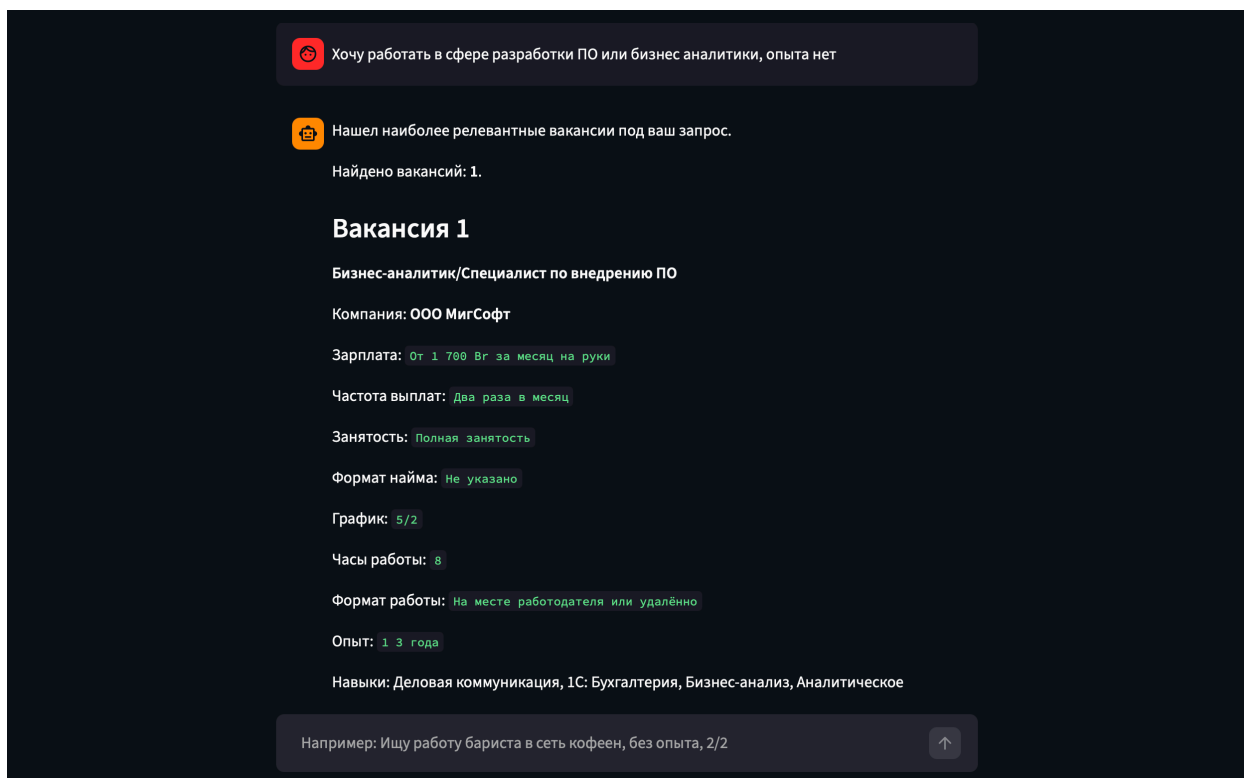


Рисунок 4.2 – Интерфейс чат-бота: запрос на старт в сфере разработки ПО или бизнес-аналитики без опыта

Примечание – Источник: собственная разработка.

На рисунке 4.3 показан запрос пользователя с правами категории В, желающего работать «на транспорте» с перевозкой людей или грузов и с пожеланием гибкого графика с почасовой оплатой. Чат-бот возвращает одну вакансию «Водитель такси (Гродно и Гродненская область)» в организации «ООО Матурхан» с разъездным форматом работы, указанием категории В в атрибутах формата, свободным графиком, вечерними или ночными сменами продолжительностью 8 или 12 часов и формулировкой «опыт не требуется». Сценарий демонстрирует согласование свободной пользовательской формулировки с конкретной профессиональной ролью и извлечёнными полями карточки вакансии.

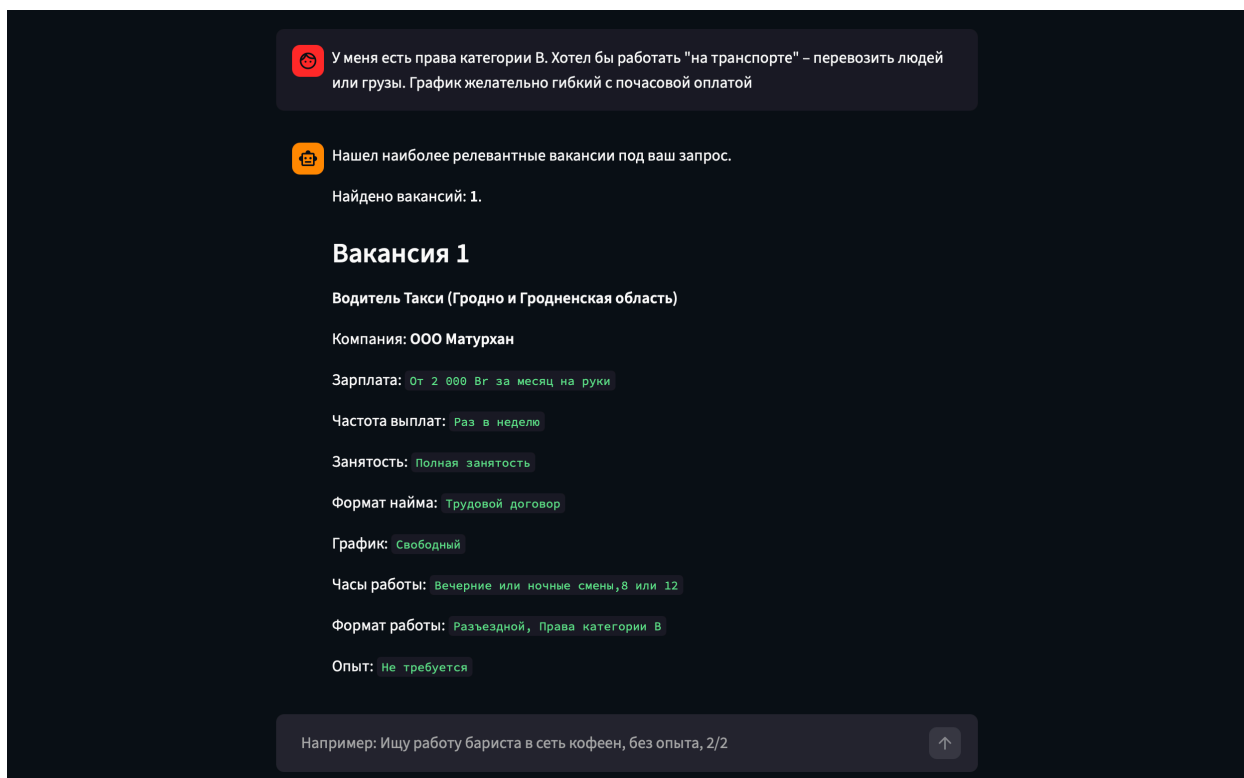


Рисунок 4.3 – Интерфейс чат-бота: поиск работы в транспорте при наличии прав категории В

Примечание – Источник: собственная разработка.

На рисунке 4.4 отражён сценарий поиска несложной удалённой работы в IT-сфере в направлениях маркетинга и копирайтинга при отсутствии опыта. Чат-бот сообщает о двух найденных вакансиях и показывает на первой позиции карточку «Специалист по подбору персонала в IT-сфере (стажёр)» в компании «ООО Брайт Эйдженси» с удалённым форматом работы, графиком «пять на два», восьмичасовым днём и формулировкой «опыт не требуется»; в поле зарплаты указан диапазон в долларах США до вычета налогов. Выдача сочетает требования пользователя к удалённому режиму и стартовой квалификации с конкретной IT-вакансией и иллюстрирует связывание разнородных ключевых слов запроса с релевантной карточкой в базе.

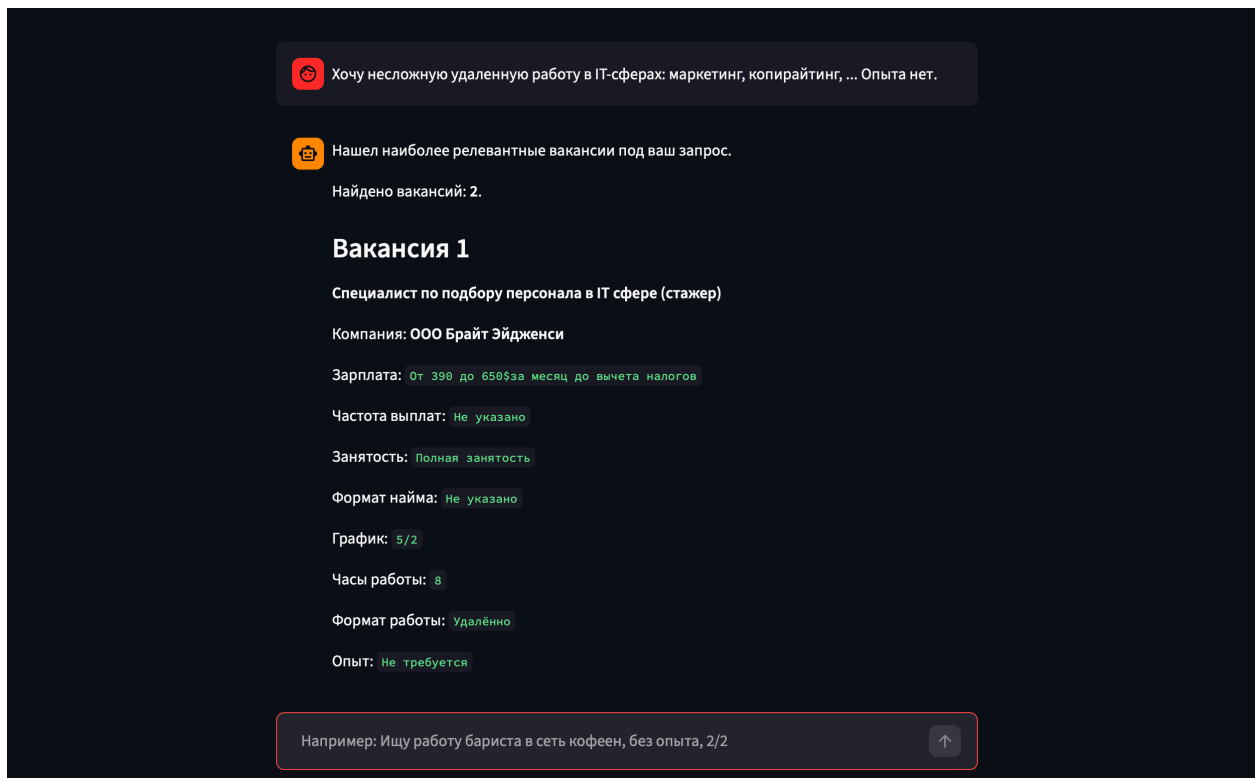


Рисунок 4.4 – Интерфейс чат-бота: запрос на удаленную стартовую работу в IT (маркетинг, копирайтинг) без опыта

Примечание – Источник: собственная разработка.

Совокупность четырёх сценариев подтверждает практическую работоспособность всех ключевых компонентов системы: серверной части на FastAPI, клиентской части на Streamlit, языковой модели Groq для извлечения структурированных фильтров, алгоритма релаксации фильтров и доменного ранжирования, а также векторного поиска в PostgreSQL с расширением pgvector. Сценарии охватывают как сценарии с явно выраженной целевой ролью, так и сценарии с неоднозначной профессиональной формулировкой и ограничениями по формату занятости.

4.3 Количественная оценка качества поиска

Контрольное тестирование полного бенчмарк-набора из шестнадцати запросов зафиксировало значения $hit@1$ равное четырнадцати из шестнадцати и $hit@5$ равное пятнадцати из шестнадцати. Это означает, что в четырнадцати сценариях из шестнадцати релевантная вакансия занимала

первую позицию выдачи, а в пятнадцати сценариях из шестнадцати релевантная вакансия попадала в первые пять позиций. В таблице 4.1 приведена сокращённая выборка из шести показательных запросов с указанием значений hit@1, hit@5 и косинусного расстояния top-1; полный протокол по всем шестнадцати запросам вынесен в приложение Б.

Таблица 4.1 – Характерные результаты контрольного тестирования

№	Запрос (сокращённо)	Топ-1: заголовок вакансии	hit@1	hit@5	Расстояние
1	Работа в пункте выдачи заказов Wildberries или Ozon, без опыта	Продавец- кассир в пункт выдачи заказов, город Гродно	да	да	0,73
3	Бариста или работа в кофейне, без опыта	Бариста (ИП Христинченко Татьяна Петровна)	да	да	0,63
6	Работа в логистике: логист, экспедитор, график 5 на 2	Менеджер по логистике (ООО К 2 Технологии)	да	да	0,28
8	Удалённая вакансия support и help desk	Support Specialist (Help Desk Analyst), GP Solutions	да	да	0,40

12	Стартовая роль в подборе персонала, IT-сфера, junior	Junior Sales Manager (ООО Сенлайн)	нет	да	0,49
16	Работа в пункте выдачи заказов Ozon или Wildberries, выдача посылок	Подходящих вакансий не найдено	нет	нет	—

Окончание таблицы 4.1

Примечание – Источник: собственная разработка.

Итоговая сводка по всему набору бенчмарков: hit@1 равен четырнадцати из шестнадцати, hit@5 равен пятнадцати из шестнадцати. Доля корректных попаданий в первую позицию составила приблизительно 0,88, доля корректных попаданий в топ-5 – приблизительно 0,94, что превышает целевое нефункциональное требование hit@5 не ниже 0,75, зафиксированное в параграфе 2.3.

Полученные значения косинусного расстояния top-1 для шестнадцати бенчмарк-запросов варьируются в широком диапазоне: от приблизительно 0,28 для семантически плотных запросов из сферы логистики до приблизительно 0,73 для запросов с обилием неформальных формулировок и брендовых наименований (пункт выдачи заказов Wildberries и Ozon). Такая вариативность является ожидаемой для bi-encoder и подтверждает, что более низкое значение косинусного расстояния top-1 коррелирует с более уверенным распознаванием целевой профессиональной роли чат-ботом.

В сокращённой выборке таблицы 4.1 присутствуют две группы характерных случаев. Первая группа – это сценарии с идеальным попаданием (hit@1 и hit@5 одновременно равны единице): запросы по пункту выдачи заказов, бариста, логистике, удалённой работе support и help desk. Вторая группа представлена сценарием № 12 (стартовая роль в подборе персонала,

IT-сфера) с `hit@1` равным нулю и `hit@5` равным единице: целевая роль присутствует в выдаче, но первой позицией выступает семантически близкая, но нерелевантная вакансия Junior Sales Manager. Сценарий № 16 (см. далее параграф 4.4) интересен тем, что демонстрирует поведение системы при заведомо узком запросе, на который в локальной базе вакансий нет ни одного подходящего предложения.

4.4 Анализ устойчивости системы и обработка ошибочных запросов

Помимо количественной оценки качества выдачи, в ходе контрольного тестирования проверялась устойчивость серверной части чат-бота к различным классам входных данных. Модель ошибок чат-бота включает четыре характерные ситуации: запрос с заведомо недостаточным числом релевантных вакансий в локальной базе, недоступность внешнего сервиса Groq, нарушение пользователем ограничений Pydantic-валидации, обращение к несуществующему URL программного интерфейса.

Сценарий № 16 контрольного тестирования демонстрирует поведение системы при запросе, для которого после применения LLM-фильтров и нескольких этапов релаксации в локальной базе не осталось ни одного кандидата, удовлетворяющего ограничению по обязательным защищённым ключам. В этой ситуации серверная часть возвращает пустой список вакансий вместе с корректно заполненными метаданными о применённых и снятых фильтрах. Клиентская часть на Streamlit отображает сообщение об отсутствии подходящих вакансий и предлагает пользователю переформулировать запрос. Тем самым обеспечивается мягкая деградация пользовательского опыта без технического сбоя.

Недоступность внешнего сервиса Groq Chat Completions имитировалась временной заменой ключа API на заведомо недопустимый. В этом случае серверная часть возбуждает контролируемое исключение, а REST-эндпоинт `GET /api/v1/search` возвращает ответ с кодом состояния 500 и единым форматом тела `ApiResponse`, содержащим поля `code`, `message` и `details`. Клиентская часть распознаёт код 500 и выводит пользователю сообщение о временной недоступности сервиса извлечения фильтров без падения интерфейса.

Нарушение пользовательских ограничений (пустая строка в параметре `query`, значение параметра `limit` за пределами диапазона от одного до пяти) приводит к ответу со статусом 422 и подробным описанием поля и типа ошибки в стандартном формате Pydantic. Обращение к несуществующему

URL программного интерфейса даёт ответ со статусом 404. В каждом из перечисленных случаев тело ответа возвращается в едином формате `ApiResponseResponse`, что упрощает разбор ошибок на клиентской стороне и при возможной интеграции чат-бота со сторонними каналами (например, Telegram-ботом из главы 5).

Качественная характеристика времени отклика наблюдалась непосредственно по ходу контрольного тестирования: на каждом из шестнадцати бенчмарк-запросов ожидание ответа от REST-эндпоинта `GET /api/v1/search` оставалось в пределах нескольких секунд и определялось преимущественно временем обращения к языковой модели Groq Cloud, тогда как векторный поиск в `pgvector` и сериализация ответа `Pydantic` в наблюдаемых условиях не вносили заметного вклада. Это качественно подтверждает выполнимость нефункционального требования по времени отклика, зафиксированного в параграфе 2.3.

4.5 Визуализация процесса обработки запроса

Для целостного представления о потоке управления внутри чат-бота построена диаграмма активности языка UML, охватывающая основной сценарий обработки пользовательского запроса от ввода в Streamlit до отображения карточек вакансий. Диаграмма отражает не только последовательность вызовов, но и развилки (например, проверка непустоты запроса, проверка достаточного числа кандидатов после релаксации) и циклы (повторное снятие фильтров до достижения порога). Соответствующая диаграмма приведена на рисунке 4.5.

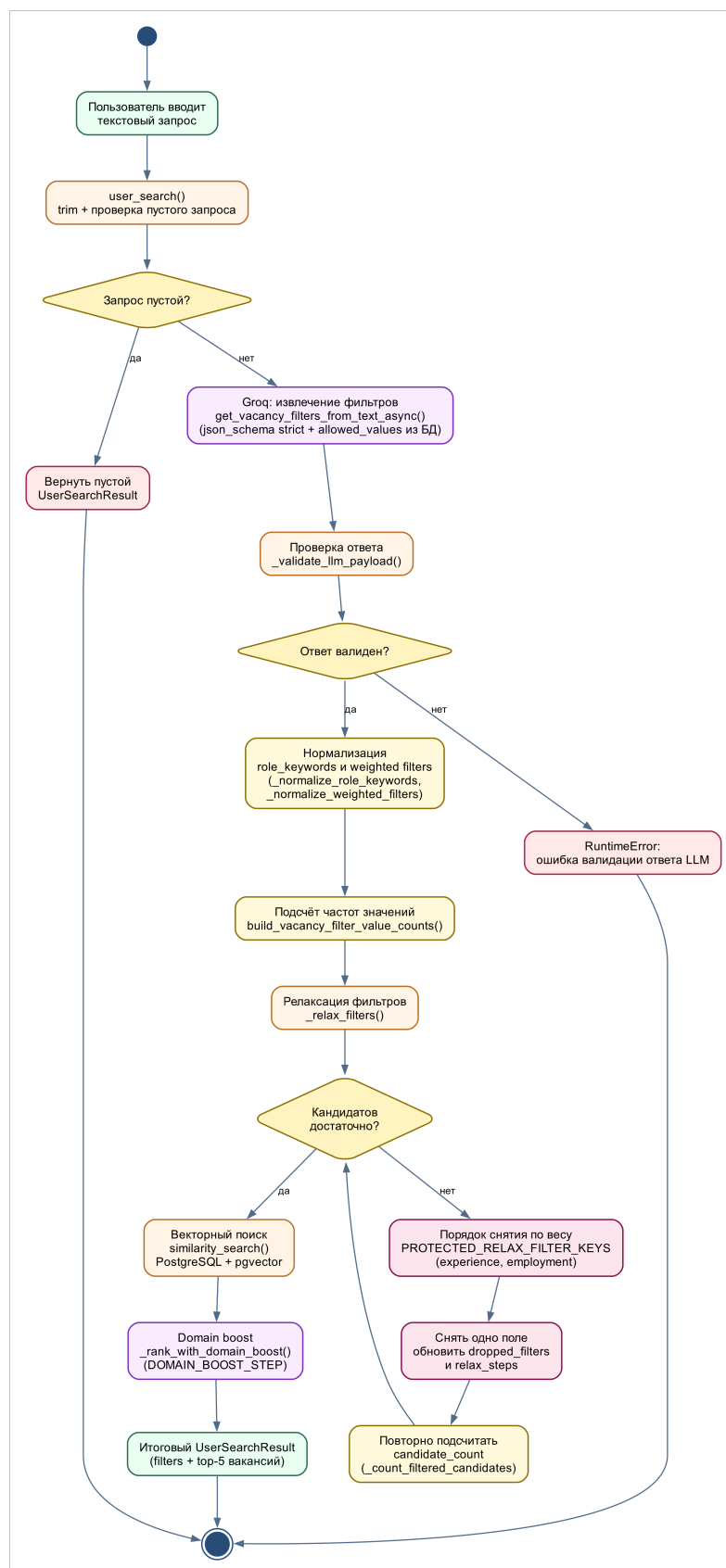


Рисунок 4.5 – Диаграмма активности UML для сценария обработки запроса чат-ботом

Примечание – Источник: собственная разработка.

Согласно рисунку 4.5, после ввода пользовательского запроса в клиентской части на Streamlit выполняется передача текста через REST-протокол на серверную часть. Серверная часть проверяет корректность запроса средствами Pydantic-валидации и обращается к языковой модели Groq для получения структурированных фильтров и ролевых ключевых слов. На основе реального распределения значений в базе вакансий (подсчитываемого оператором `SELECT DISTINCT`) оценивается число потенциальных кандидатов; при недостаточном их числе активируется алгоритм релаксации фильтров с защитой приоритетных ключей.

После достижения порогового числа кандидатов выполняется векторный поиск по полю `embedding` с использованием оператора косинусного расстояния расширения `pgvector`. К полученной выборке применяется доменное ранжирование, учитывающее совпадения по ролевым ключевым словам и встроенным доменным словарям. На выходе формируется итоговый ответ из пяти вакансий вместе с метаданными о применённых и снятых фильтрах, который сериализуется в формат JSON и передаётся клиентской части для отображения карточек в чате. Описанная диаграмма используется далее как опорный артефакт для главы 5 при формулировании направлений развития чат-бота.

4.6 Выводы по главе 4

1. На наборе из шестнадцати эталонных бенчмарк-запросов чат-бот продемонстрировал значение метрики `hit@1`, равное четырнадцати из шестнадцати, и значение метрики `hit@5`, равное пятнадцати из шестнадцати, что превышает целевой нефункциональный показатель `hit@5` не ниже 0,75, зафиксированный в параграфе 2.3.
2. Функциональное тестирование четырёх типовых пользовательских сценариев (работа в пункте выдачи заказов, стажировка в сфере информационных технологий, удалённая работа интернет-маркетолога, работа в кофейне) подтвердило корректную работу всех ключевых компонентов чат-бота: клиентской части на Streamlit, серверной части на FastAPI, извлечения структурированных фильтров языковой моделью Groq, алгоритмов релаксации и доменного ранжирования, а также векторного поиска в PostgreSQL с расширением `pgvector`.
3. Анализ устойчивости показал, что чат-бот корректно обрабатывает четыре характерных класса нештатных ситуаций: запрос без подходящих вакансий в локальной базе (мягкая деградация выдачи),

недоступность внешнего сервиса Groq (статус 500 и единый формат `ApiResponse`), нарушение Pydantic-валидации параметров запроса (статус 422), обращение к несуществующему URL (статус 404).

4. Качественное наблюдение за временем отклика по ходу тестирования подтвердило выполнимость нефункционального требования параграфа 2.3: ожидание ответа REST-эндпоинта `GET /api/v1/search` оставалось в пределах нескольких секунд и определялось преимущественно временем обращения к языковой модели Groq Cloud.
5. Построенная диаграмма активности UML визуализирует поток управления чат-бота с явным выделением развилок и циклов и используется в качестве опорного артефакта при формулировании направлений дальнейшего развития программного продукта в главе 5.

5 НАПРАВЛЕНИЯ ДАЛЬНЕЙШЕГО РАЗВИТИЯ ПРОЕКТА

Пятая глава завершает основной текст курсовой работы и посвящена перспективам развития программного комплекса интеллектуального поиска вакансий с диалоговым интерфейсом. Цель главы – системно сопоставить уже реализованную архитектуру из главы 3 и результаты экспериментальной проверки из главы 4 с возможными шагами по расширению функциональности, масштабированию и повышению качества поиска. Направления сгруппированы таким образом, чтобы отделить прикладные шаги продуктового развития от исследовательской траектории, связанной с усилением моделей извлечения и переранжирования.

5.1 Масштабирование и продуктивизация

Для перевода прототипа в устойчиво эксплуатируемый контур целесообразна контейнеризация компонентов системы с описанием сервисов в составе `docker-compose`: сервер баз данных PostgreSQL с расширением `pgvector`, серверная часть на базе FastAPI и клиентская часть на базе Streamlit. Такой состав повторяет логику трёхуровневой архитектуры и упрощает воспроизводимое развёртывание на учебном стенде или в небольшой инфраструктуре карьерного центра высшего учебного заведения Республики Беларусь.

На этапе роста объёма базы знаний разумно запланировать переход к индексу приближённого поиска ближайших соседей в семантике графа иерархической навигации в малом мире (HNSW) средствами расширения `pgvector`: на корпусах большого объёма это даёт заметное ускорение по сравнению с полным перебором при сохранении приемлемой точности отбора кандидатов.

Отдельным блоком продуктового развития является организация периодического обновления данных с портала `rabota.by` по расписанию, контроль версий схемы базы данных средствами Alembic и регламентное офлайн-тестирование эталонных запросов из модуля количественной оценки поиска. По результатам тестирования возможно отслеживать стабильность метрик качества поиска `hit` на первую позицию и `hit` на пять первых позиций и своевременно реагировать на деградацию из-за изменений состава корпуса или поведения внешнего сервиса большой языковой модели.

Дополнительно целесообразно внедрить конвейер непрерывной интеграции в системе контроля версий: статический анализ кода, автоматические тесты и сборка контейнерных образов. Это снижает риск регрессий при доработках серверной части и клиентской части и приближает проект к промышленным практикам разработки.

5.2 Интеграция с популярными мессенджерами

Уже реализованный программный интерфейс REST для эндпоинта получения поисковой выдачи по запросу пользователя создаёт естественную точку расширения: достаточно добавить легковесный интерфейс для Telegram Bot API (режим long polling или webhook), который транслирует сообщения диалога в параметры запроса к серверной части и форматирует карточки вакансий под ограничения мессенджера.

Интеграция с Telegram особенно хорошо сочетается с темой курсовой работы, так как диалоговый канал совпадает с привычным для соискателей способом общения и не требует установки отдельного веб-приложения. При необходимости перспективным следующим шагом можно считать поддержку дополнительных каналов (например, экосистемы VK или мессенджера Viber), если учитывать регламенты доступа и политики платформ.

5.3 Реализация персонализации

Текущая реализация чат-бота опирается на локальное состояние сессии веб-клиента Streamlit и не сохраняет долговременный профиль пользователя. Направление персонализации предполагает учёт предпочтений и откликов пользователя в пределах длительного диалога: накопление подтверждённых фильтров, корректировку весов по обратной связи под карточками вакансий и список избранных позиций для повторного просмотра.

Отдельным улучшением является загрузка файла резюме в формате PDF или DOCX с извлечением текста и построением векторного представления резюме без обязательной вербализации опыта пользователем в чате. Такой сценарий снижает когнитивную нагрузку на соискателя и дополняет семантический поиск по корпусу вакансий.

5.4 Внедрение генеративного слоя для резюмирования вакансий

В реализованном контуре большая языковая модель выполняет структурирование пользовательского запроса и извлечение фильтров, а итоговая выдача формируется как список карточек вакансий после поиска и доменного ранжирования. Следующий логический шаг – добавить генеративный слой после извлечения кандидатов: для нескольких лучших позиций формировать краткое объяснение, почему вакансия может подходить именно данному пользователю, выделять потенциальные сильные и слабые стороны соответствия и при необходимости предлагать сравнение вакансий в табличном виде.

Такое расширение приближает систему к классической схеме retrieval-augmented generation (дополнение генерации извлечением), в которой после поиска по базе знаний включается генеративная модель для интерпретации найденных документов. Технически новый функционал может быть оформлен отдельным методом серверной части и отдельным маршрутом программного интерфейса, чтобы не увеличивать задержку основного запроса поиска для пользователей, которым нужна только выдача карточек.

5.5 Усиление слоя извлечения кандидатов (исследовательское направление)

Эксперименты главы 4 показывают сильные стороны двунаправленного энкодера семейства Sentence-BERT при офлайн-индексации корпуса, но также фиксируют случаи, где семантически близкие, тем не менее нерелевантные предметной области вакансии могут конкурировать с целевыми позициями. Исследовательское направление развития связано с постановкой дополнительного этапа переранжирования на базе cross-encoder для ограниченного списка кандидатов, дообучением компактной модели MiniLM на материале корпуса вакансий Республики Беларусь и гибридизацией плотного векторного поиска с лексическими методами полнотекстового поиска.

Эти шаги типичны для научных продолжений работы: они увеличивают вычислительную стоимость и сложность сопровождения, но дают потенциальный прирост точности на коротких запросах и в узких профессиональных нишах. Поэтому целесообразно рассматривать их как

этап отдельного исследования после стабилизации продуктового контура и накопления обратной связи пользователей.

5.6 Выводы по главе 5

1. Сформулированы два пласта направлений развития: продуктовый (контейнеризация, интеграция с мессенджерами, персонализация, генеративное резюмирование вакансий) и исследовательский (cross-encoder, дообучение энкодера, гибридный поиск).
2. Интеграция с Telegram определена как наиболее прямое расширение канала доставки сервиса при сохранении уже реализованного программного интерфейса серверной части.
3. Генеративное резюмирование топовых вакансий обозначено как путь к более полной реализации парадигмы дополнения генерации извлечением при сохранении управляемости задержки ответа.
4. Усиление слоя извлечения и переранжирования отнесено к перспективам самостоятельного исследования после завершения инженерной зрелости прототипа.

ЗАКЛЮЧЕНИЕ

1. Систематизированы теоретические основы построения чат-ботов и семантического поиска вакансий; обоснована применимость двунаправленного энкодера семейства Sentence-BERT и элементов парадигмы дополнения генерации извлечением (retrieval-augmented generation) для задачи подбора вакансий на диалоговом интерфейсе (параграфы 1.1–1.4).
2. Проанализированы специфика рынка труда Республики Беларусь, существующие диалоговые системы и программные средства поиска вакансий; выявлены ограничения сочетания жёстких фильтров с лексическим поиском и сформулирована постановка задачи с функциональными и нефункциональными требованиями (параграфы 2.1–2.3).
3. Обоснован выбор программного стека: язык Python, серверная часть на FastAPI, клиент Streamlit, PostgreSQL с расширением pgvector, мультязычная модель представления предложений MiniLM, облачный сервис большой языковой модели Groq (параграф 2.4).
4. Спроектирована трёхуровневая клиент-серверная архитектура с протоколом REST и реализована схема базы знаний с векторным полем размерности 384 и миграциями Alembic (параграфы 3.1–3.2).
5. Реализован серверный программный интерфейс на FastAPI с маршрутом GET /api/v1/search, проверкой входных данных средствами Pydantic, автоматически формируемой документацией OpenAPI и единым форматом сообщений об ошибках (параграф 3.3).
6. Реализована логика большой языковой модели: извлечение структурированных фильтров по запросу пользователя с использованием режима json_schema и строгой валидации на стороне сервера (параграф 3.4).
7. Реализованы адаптивная релаксация фильтров при недостаточном числе кандидатов с защитой приоритетных признаков и доменное ранжирование выдачи как упрощённая альтернатива дорогостоящему переранжированию cross-encoder (параграф 3.5).
8. Разработан веб-интерфейс на Streamlit с историей диалога и карточками вакансий; настроен контур сбора данных с портала rabota.by и взвешенное текстовое представление вакансий для векторизации (параграфы 3.6–3.7).
9. На наборе из шестнадцати эталонных бенчмарк-запросов зафиксированы значения метрики hit@1, равные четырнадцати из

шестнадцати, и метрики `hit@5`, равные пятнадцати из шестнадцати; функциональное тестирование пользовательских сценариев и оценка времени отклика программного интерфейса подтвердили работоспособность комплекса в целом (параграфы 4.1–4.5).

10. Предложены направления дальнейшего развития: контейнеризация и регламент обновления данных, интеграция с мессенджерами, персонализация и генеративное резюмирование вакансий, а также исследовательские шаги по усилению слоя извлечения и переранжирования (глава 5).

Таким образом, цель работы достигнута, все поставленные задачи выполнены: разработан работоспособный чат-бот для семантического поиска вакансий на рынке труда Республики Беларусь, реализующий клиент-серверную архитектуру и интеграцию с большой языковой моделью.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Vaswani A. et al. Attention Is All You Need // Advances in Neural Information Processing Systems : proceedings. – 2017. – Vol. 30.
2. Devlin J. et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding // Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics. – 2019. – P. 4171–4186.
3. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. – 2019.
4. Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks // Advances in Neural Information Processing Systems : proceedings. – 2020. – Vol. 33.
5. Wang W. et al. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers // Advances in Neural Information Processing Systems : proceedings. – 2020. – Vol. 33.
6. PostgreSQL Documentation [Электронный ресурс] / PostgreSQL Global Development Group. – Режим доступа: <https://www.postgresql.org/docs/current/>. – Дата обращения: 09.05.2026.
7. pgvector: Open-source vector similarity search for Postgres [Электронный ресурс]. – Режим доступа: <https://github.com/pgvector/pgvector>. – Дата обращения: 09.05.2026.
8. FastAPI Documentation [Электронный ресурс] / Sebastián Ramírez. – Режим доступа: <https://fastapi.tiangolo.com/>. – Дата обращения: 09.05.2026.
9. Streamlit Documentation [Электронный ресурс] / Snowflake Inc. – Режим доступа: <https://docs.streamlit.io/>. – Дата обращения: 09.05.2026.
10. SQLAlchemy 2.x Documentation (ORM, ядро async-драйверов; см. также asyncpg) [Электронный ресурс]. – Режим доступа: <https://docs.sqlalchemy.org/en/20/>; <https://magicstack.github.io/asyncpg/current/>. – Дата обращения: 09.05.2026.
11. Pydantic Documentation [Электронный ресурс]. – Режим доступа: <https://docs.pydantic.dev/latest/>. – Дата обращения: 09.05.2026.
12. Sentence-Transformers Documentation [Электронный ресурс]. – Режим доступа: <https://www.sbert.net/>. – Дата обращения: 09.05.2026.
13. Groq – Chat Completions API Reference [Электронный ресурс] / Groq, Inc. – Режим доступа: <https://console.groq.com/docs/api-reference#chat-create>. – Дата обращения: 09.05.2026.

14. paraphrase-multilingual-MiniLM-L12-v2 [Электронный ресурс] / Hugging Face, Inc. – Режим доступа: <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>. – Дата обращения: 09.05.2026.

15. rabota.by – портал поиска вакансий в Республике Беларусь [Электронный ресурс]. – Режим доступа: <https://rabota.by/>. – Дата обращения: 09.05.2026.

16. Telegram Bot API Documentation [Электронный ресурс] / Telegram Messenger LLP. – Режим доступа: <https://core.telegram.org/bots/api>. – Дата обращения: 09.05.2026.

17. GitHub [Электронный ресурс] / Проект автора на GitHub. – Режим доступа: <https://github.com/Daniello0/ai-job-finder-python>.

ПРИЛОЖЕНИЕ А

Листинги ключевых модулей программного комплекса

```
from __future__ import annotations

from sqlalchemy import String, Text, UniqueConstraint
from sqlalchemy.orm import DeclarativeBase, Mapped,
mapped_column

try:
    from pgvector.sqlalchemy import Vector
except ModuleNotFoundError: # pragma: no cover
    Vector = None # type: ignore[assignment]

class Base(DeclarativeBase):
    """Base class for ORM models."""

class Vacancy(Base):
    """Vacancy ORM model used for ingestion and semantic
    search."""

    __tablename__ = "vacancies"
    __table_args__ = (UniqueConstraint("url",
name="uq_vacancies_url"),)

    id: Mapped[int] = mapped_column(primary_key=True,
autoincrement=True)

    title: Mapped[str] = mapped_column(String(512),
nullable=False)
    company: Mapped[str] = mapped_column(String(512),
nullable=False)
    salary: Mapped[str] = mapped_column(String(256),
nullable=False)
    payment_frequency: Mapped[str] = mapped_column(String(128),
nullable=False)
    experience: Mapped[str] = mapped_column(String(256),
nullable=False)
    employment: Mapped[str] = mapped_column(String(256),
nullable=False)
    hiring_format: Mapped[str] = mapped_column(String(256),
nullable=False)
    schedule: Mapped[str] = mapped_column(String(256),
nullable=False)
    hours: Mapped[str] = mapped_column(String(256),
nullable=False)
    work_format: Mapped[str] = mapped_column(String(256),
nullable=False)

    skills: Mapped[str] = mapped_column(Text, nullable=False)
    url: Mapped[str] = mapped_column(String(2048),
nullable=False)
    description: Mapped[str] = mapped_column(Text,
nullable=False)
```

```

    if Vector is not None:
        embedding: Mapped[list[float] | None] = mapped_column(
            Vector(384), nullable=True
        )
    else:
        embedding: Mapped[str | None] = mapped_column(Text,
            nullable=True)

```

Листинг А.1 – Файл backend/src/features/database/models.py

Примечание – Источник: собственная разработка.

```

"""FastAPI router for vacancy search endpoints."""

from __future__ import annotations

from dataclasses import asdict

from fastapi import APIRouter, HTTPException, Query
from pydantic import BaseModel, HttpUrl

from common.constants.api import (
    API_V1_PREFIX,
    ERROR_MESSAGE_BLANK_QUERY,
    ERROR_MESSAGE_SEARCH_UNAVAILABLE,
    HTTP_BAD_REQUEST,
    HTTP_NOT_FOUND,
    HTTP_INTERNAL_SERVER_ERROR,
    HTTP_UNPROCESSABLE_ENTITY,
    MAX_VACANCY_RESULTS,
    QUERY_MIN_LENGTH,
    SEARCH_ENDPOINT,
    SEARCH_TAG,
)
from common.schemas.api_error import ApiErrorResponse
from features.search.service import user_search

router = APIRouter(prefix=API_V1_PREFIX, tags=[SEARCH_TAG])

class VacancySearchResponse(BaseModel):
    """Public vacancy payload returned to the frontend."""

    id: int
    title: str
    company: str
    salary: str
    payment_frequency: str
    experience: str
    employment: str
    hiring_format: str
    schedule: str
    hours: str
    work_format: str
    skills: str
    url: HttpUrl

```

```

description: str
cosine_distance: float

@router.get(
    SEARCH_ENDPOINT,
    response_model=list[VacancySearchResponse],
    responses={
        HTTP_BAD_REQUEST: {
            "model": ApiErrorResponse,
            "description": "Invalid request payload.",
        },
        HTTP_NOT_FOUND: {
            "model": ApiErrorResponse,
            "description": "Endpoint not found.",
        },
        HTTP_UNPROCESSABLE_ENTITY: {
            "model": ApiErrorResponse,
            "description": "Validation failed.",
        },
        HTTP_INTERNAL_SERVER_ERROR: {
            "model": ApiErrorResponse,
            "description": "Internal server error.",
        },
    },
)

async def search_vacancies(
    query: str = Query(
        ...,
        min_length=QUERY_MIN_LENGTH,
        description="User profile or free-form query for vacancy
matching.",
    ),
    limit: int = Query(
        default=MAX_VACANCY_RESULTS,
        ge=1,
        le=MAX_VACANCY_RESULTS,
        description="Maximum number of vacancies to return.",
    ),
) -> list[VacancySearchResponse]:
    """Return up to five most relevant vacancies with full
details."""

    normalized_query = query.strip()
    if not normalized_query:
        raise HTTPException(
            status_code=HTTP_BAD_REQUEST,
            detail=ERROR_MESSAGE_BLANK_QUERY
        )
    try:
        search_result = await user_search(normalized_query,
limit=limit)
    except RuntimeError as error:
        raise HTTPException(
            status_code=HTTP_INTERNAL_SERVER_ERROR,
            detail=ERROR_MESSAGE_SEARCH_UNAVAILABLE,
        ) from error
    return [
        VacancySearchResponse.model_validate(asdict(vacancy))

```

```

        for vacancy in search_result.vacancies
    ]

```

Листинг А.2 – Файл backend/src/features/search/router.py

Примечание – Источник: собственная разработка.

```

"""Schemas for orchestration search flow: LLM filters +
embedding ranking."""

from __future__ import annotations

from dataclasses import dataclass

from features.embedding.schemas import SimilaritySearchResult

@dataclass(frozen=True, slots=True)
class FilterRelaxStep:
    """One guardrail step where a single filter value is
    removed."""

    step: int
    filter_key: str
    filter_value: str
    removed_weight: float
    value_count_in_db: int
    candidates_before: int
    candidates_after: int

@dataclass(frozen=True, slots=True)
class UserSearchResult:
    """Final user search output with inferred filters and ranked
    vacancies."""

    user_query: str
    role_keywords: list[str]
    llm_filters: dict[str, list[dict[str, float | str]]]
    all_value_counts: dict[str, dict[str, int]]
    selected_value_counts: dict[str, dict[str, int]]
    applied_filters: dict[str, list[str]]
    dropped_filters: dict[str, list[str]]
    relax_steps: list[FilterRelaxStep]
    candidate_count: int
    vacancies: list[SimilaritySearchResult]

```

Листинг А.3 – Файл backend/src/features/search/schemas.py

Примечание – Источник: собственная разработка.

```

"""Groq service helpers for extracting vacancy filters from user
text."""

```



```

from __future__ import annotations

import asyncio
import json
import os
from typing import Any

import requests
from sqlalchemy import func, select

from common.constants.llm import (
    VACANCY_OPTIONAL_KEYS,
    GROQ_BASE_URL,
    GROQ_MODEL,
    GROQ_TIMEOUT_SECONDS,
    VACANCY_FILTER_KEYS,
    VACANCY_FILTERS_CONTEXT_TEMPLATE,
    VACANCY_FILTERS_SYSTEM_PROMPT_TEMPLATE,
)
from features.database.db import async_session_factory
from features.database.models import Vacancy
from features.llm.schemas import LlmAnswer

_FILTER_COLUMN_MAP = {
    "payment_frequency": Vacancy.payment_frequency,
    "experience": Vacancy.experience,
    "employment": Vacancy.employment,
    "hiring_format": Vacancy.hiring_format,
    "schedule": Vacancy.schedule,
    "hours": Vacancy.hours,
    "work_format": Vacancy.work_format,
}

def _get_groq_api_key() -> str:
    api_key = os.getenv("GROQ_API_KEY")
    if api_key and api_key.strip():
        return api_key.strip()
    msg = "GROQ_API_KEY is missing. Add it to environment variables."
    raise ValueError(msg)

def _build_json_schema(allowed_values: dict[str, list[str]]) -> dict[str, Any]:
    item_schema_by_key: dict[str, dict[str, Any]] = {}
    for key in VACANCY_FILTER_KEYS:
        item_schema_by_key[key] = {
            "type": "object",
            "properties": {
                "value": {"type": "string", "enum": allowed_values[key]},
                "weight": {"type": "number", "minimum": 0, "maximum": 1},
            },
            "required": ["value", "weight"],
            "additionalProperties": False,
        }

```

```

properties: dict[str, Any] = {
    key: {
        "type": "array",
        "items": item_schema_by_key[key],
        "maxItems": 2,
    }
    for key in VACANCY_FILTER_KEYS
}
properties["role_keywords"] = {
    "type": "array",
    "items": {"type": "string"},
    "maxItems": 5,
}
return {
    "type": "object",
    "properties": properties,
    "required": [*VACANCY_FILTER_KEYS, "role_keywords"],
    "additionalProperties": False,
}

def _build_payload(
    prompt: str, context: str, allowed_values: dict[str,
list[str]]
) -> dict[str, Any]:
    return {
        "model": GROQ_MODEL,
        "temperature": 0,
        "response_format": {
            "type": "json_schema",
            "json_schema": {
                "name": "vacancy_filters",
                "strict": True,
                "schema": _build_json_schema(allowed_values),
            },
        },
        "messages": [
            {"role": "system", "content": prompt},
            {"role": "user", "content": context},
        ],
    }

def _extract_content(data: dict[str, Any]) -> str:
    choices = data.get("choices")
    if not isinstance(choices, list) or not choices:
        raise ValueError("Groq response does not contain
choices.")
    message = choices[0].get("message")
    if not isinstance(message, dict):
        raise ValueError("Groq response message is missing.")
    content = message.get("content")
    if not isinstance(content, str) or not content.strip():
        raise ValueError("Groq response content is empty.")
    return content.strip()

def _strip_code_fences(text: str) -> str:
    cleaned = text.strip()
    if cleaned.startswith("`"):

```

```

        cleaned = cleaned[3:]
        if cleaned.startswith("json"):
            cleaned = cleaned[4:]
        cleaned = cleaned.strip()
        if cleaned.endswith("`"):
            cleaned = cleaned[:-3]
    return cleaned.strip()

def _parse_json_payload(raw_content: str) -> dict[str, Any]:
    normalized = _strip_code_fences(raw_content)
    parsed = json.loads(normalized)
    if not isinstance(parsed, dict):
        raise ValueError("LLM output must be a JSON object.")
    return parsed

def _empty_filter_object() -> dict[str, list[str]]:
    return {key: [] for key in VACANCY_FILTER_KEYS}

def _empty_filter_count_object() -> dict[str, dict[str, int]]:
    return {key: {} for key in VACANCY_FILTER_KEYS}

def _build_system_prompt(allowed_values: dict[str, list[str]])
-> str:
    allowed_values_json = json.dumps(allowed_values,
ensure_ascii=False)
    return VACANCY_FILTERS_SYSTEM_PROMPT_TEMPLATE.format(
        allowed_values_json=allowed_values_json
    )

def _normalize_db_value(value: str | None) -> str | None:
    if value is None:
        return None
    normalized = value.strip()
    return normalized or None

def _validate_llm_payload(
    payload: dict[str, Any], allowed_values: dict[str,
list[str]]
) -> list[str]:
    errors: list[str] = []
    required = set(VACANCY_FILTER_KEYS)
    optional = set(VACANCY_OPTIONAL_KEYS)
    payload_keys = set(payload.keys())
    if not required.issubset(payload_keys):
        errors.append("JSON must contain all required filter
keys.")
    unknown = sorted(payload_keys - required - optional)
    if unknown:
        errors.append(f"JSON contains unknown keys: {'',
'.join(unknown)}")

    role_keywords = payload.get("role_keywords", [])
    if not isinstance(role_keywords, list):
        errors.append("Field 'role_keywords' must be an array.")

```

```

        elif not all(isinstance(item, str) for item in
role_keywords):
            errors.append("Field 'role_keywords' must contain only
strings.")
        elif len(role_keywords) > 5:
            errors.append("Field 'role_keywords' must contain at
most 5 items.")
        for key in VACANCY_FILTER_KEYS:
            value = payload.get(key)
            if not isinstance(value, list):
                errors.append(f"Field '{key}' must be an array.")
                continue
            allowed_set = set(allowed_values[key])
            invalid_values: list[str] = []
            for item in value:
                if not isinstance(item, dict):
                    errors.append(f"Field '{key}' items must be
objects.")
                    continue
                if set(item.keys()) != {"value", "weight"}:
                    errors.append(
                        f"Field '{key}' items must contain only
'value' and 'weight'."
                    )
                    continue
                raw_filter_value = item.get("value")
                raw_weight = item.get("weight")
                if not isinstance(raw_filter_value, str):
                    errors.append(f"Field '{key}' item 'value' must
be a string.")
                    continue
                if not isinstance(raw_weight, int | float):
                    errors.append(f"Field '{key}' item 'weight' must
be numeric.")
                    continue
                if not 0 <= float(raw_weight) <= 1:
                    errors.append(f"Field '{key}' item 'weight' must
be in range [0, 1].")
                    continue
                if raw_filter_value not in allowed_set:
                    invalid_values.append(raw_filter_value)
            if invalid_values:
                joined = ", ".join(invalid_values)
                errors.append(f"Field '{key}' has invalid values:
{joined}.")
        return errors

def _response_error_message(response: requests.Response) -> str:
    try:
        data = response.json()
        if isinstance(data, dict):
            error = data.get("error")
            if isinstance(error, dict) and
isinstance(error.get("message"), str):
                return error["message"]
            if isinstance(error, str):
                return error
    except ValueError:
        pass

```

```

        return response.text.strip() or "Unknown Groq error."

def get_llm_answer_service(
    prompt: str, context: str, allowed_values: dict[str,
list[str]]
) -> LlmAnswer:
    """Call Groq chat completions and parse JSON from the first
choice."""
    api_key = _get_groq_api_key()
    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json",
    }
    payload = _build_payload(prompt, context, allowed_values)
    try:
        response = requests.post(
            GROQ_BASE_URL,
            headers=headers,
            json=payload,
            timeout=GROQ_TIMEOUT_SECONDS,
        )
    except requests.RequestException as err:
        raise RuntimeError(f"Groq request failed: {err!r}") from
err
    if not response.ok:
        details = _response_error_message(response)
        msg = f"Groq request failed ({response.status_code}):
{details}"
        raise RuntimeError(msg)
    data = response.json()
    raw_content = _extract_content(data)
    payload = _parse_json_payload(raw_content)
    model = str(data.get("model", GROQ_MODEL))
    return LlmAnswer(model=model, raw_content=raw_content,
payload=payload)

async def build_vacancy_filter_allowed_values() -> dict[str,
list[str]]:
    """Build allowed filter values from unique vacancy fields in
PostgreSQL."""
    counts = await build_vacancy_filter_value_counts()
    return {key: sorted(counts[key]) for key in
VACANCY_FILTER_KEYS}

async def build_vacancy_filter_value_counts() -> dict[str,
dict[str, int]]:
    """Build ``value -> count`` maps for each filter field from
PostgreSQL."""
    allowed_values = _empty_filter_count_object()
    async with async_session_factory() as session:
        for key in VACANCY_FILTER_KEYS:
            column = _FILTER_COLUMN_MAP[key]
            stmt = (
                select(column, func.count(column))
                .where(column.isnot(None))
                .group_by(column)
                .order_by(column)

```

```

        )
        result = await session.execute(stmt)
        value_counts: dict[str, int] = {}
        for value, count in result.all():
            normalized = _normalize_db_value(value)
            if normalized is not None:
                value_counts[normalized] = int(count)
        allowed_values[key] = value_counts
    return allowed_values

async def get_vacancy_filters_from_text_async(user_message: str)
-> dict[str, Any]:
    """Build DB-based prompt, request filters from LLM, and
    validate response."""
    allowed_values = await build_vacancy_filter_allowed_values()
    prompt = _build_system_prompt(allowed_values)
    context = VACANCY_FILTERS_CONTEXT_TEMPLATE.format(
        user_message=user_message.strip()
    )
    answer = get_llm_answer_service(prompt, context,
allowed_values)
    errors = _validate_llm_payload(answer.payload,
allowed_values)
    if not errors:
        return answer.payload
    details = "; ".join(errors) if errors else "Unknown
validation error."
    raise RuntimeError(f"LLM response validation failed:
{details}")

def get_vacancy_filters_from_text(user_message: str) ->
dict[str, Any]:
    """Synchronous wrapper for LLM-based vacancy filter
    extraction."""
    try:
        asyncio.get_running_loop()
    except RuntimeError:
        return
    asyncio.run(get_vacancy_filters_from_text_async(user_message))
    msg = "Use get_vacancy_filters_from_text_async() inside
    async context."
    raise RuntimeError(msg)

```

Листинг А.4 – Файл backend/src/features/llm/service.py

Примечание – Источник: собственная разработка.

```

"""Orchestrate user search: LLM filter extraction then filtered
embedding search."""

from __future__ import annotations

import asyncio
from typing import Any

```

```

from common.constants.embedding import DEFAULT_SIMILARITY_TOP_K
from common.constants.llm import VACANCY_FILTER_KEYS
from common.constants.search import (
    DEFAULT_MAX_VALUES_PER_FILTER,
    DEFAULT_MIN_FILTERED_CANDIDATES,
    DOMAIN_BOOST_STEP,
    PROTECTED_RELAX_FILTER_KEYS,
    PROTECTED_RELAX_WEIGHT_PENALTY,
    ROLE_KEYWORDS_HARD_LOCK,
)
from sqlalchemy import Select, func, or_, select

from features.database.db import async_session_factory
from features.database.models import Vacancy
from features.embedding.schemas import SimilaritySearchResult
from features.embedding.similarity_search_service import
similarity_search
from features.llm.service import (
    build_vacancy_filter_value_counts,
    get_vacancy_filters_from_text_async,
)
from features.search.schemas import FilterRelaxStep,
UserSearchResult

_FILTER_COLUMN_MAP = {
    "payment_frequency": Vacancy.payment_frequency,
    "experience": Vacancy.experience,
    "employment": Vacancy.employment,
    "hiring_format": Vacancy.hiring_format,
    "schedule": Vacancy.schedule,
    "hours": Vacancy.hours,
    "work_format": Vacancy.work_format,
}

_DOMAIN_KEYWORDS = {
    "horeca": ("бариста", "кофе", "кофейн", "десерт",
"кондитер", "пекар", "кухн"),
    "it": ("python", "backend", "developer", "разработ",
"программист", "fastapi"),
    "logistics": ("склад", "логист", "комплектов", "кладов",
"грузчик"),
}

def _empty_filters() -> dict[str, list[str]]:
    return {key: [] for key in VACANCY_FILTER_KEYS}

def _empty_weighted_filters() -> dict[str, list[dict[str, float
| str]]]:
    return {key: [] for key in VACANCY_FILTER_KEYS}

def _clone_non_empty(filters: dict[str, list[str]]) -> dict[str,
list[str]]:
    return {key: list(values) for key, values in filters.items()
if values}

def _empty_filter_counts() -> dict[str, dict[str, int]]:

```

```

    return {key: {} for key in VACANCY_FILTER_KEYS}

def _with_all_count_keys(
    counts: dict[str, dict[str, int]],
) -> dict[str, dict[str, int]]:
    completed = _empty_filter_counts()
    completed.update(counts)
    return completed

def _with_all_keys(filters: dict[str, list[str]]) -> dict[str,
list[str]]:
    completed = _empty_filters()
    completed.update(filters)
    return completed

def _normalize_weighted_filters(
    payload: dict[str, Any],
) -> dict[str, list[tuple[str, float]]]:
    normalized: dict[str, list[tuple[str, float]]] = {
        key: [] for key in VACANCY_FILTER_KEYS
    }
    for key in VACANCY_FILTER_KEYS:
        items = payload.get(key, [])
        if not isinstance(items, list):
            msg = f"Filter '{key}' must be a list of objects."
            raise ValueError(msg)
        weight_by_value: dict[str, float] = {}
        for item in items:
            if not isinstance(item, dict):
                continue
            raw_value = item.get("value")
            raw_weight = item.get("weight")
            if not isinstance(raw_value, str) or not
raw_value.strip():
                continue
            if not isinstance(raw_weight, int | float):
                continue
            value = raw_value.strip()
            weight = float(raw_weight)
            weight_by_value[value] =
max(weight_by_value.get(value, 0.0), weight)
            normalized[key] = sorted(
                ((value, weight) for value, weight in
weight_by_value.items()),
                key=lambda item: item[1],
                reverse=True,
           )[:DEFAULT_MAX_VALUES_PER_FILTER]
    return normalized

def _serialize_weighted_filters(
    filters: dict[str, list[tuple[str, float]]],
) -> dict[str, list[dict[str, float | str]]]:
    result = _empty_weighted_filters()
    for key in VACANCY_FILTER_KEYS:
        result[key] = [
            {"value": value, "weight": round(weight, 4)}

```



```

        for value, weight in filters[key]
    ]
    return result

def _plain_filters_from_weighted(
    filters: dict[str, list[tuple[str, float]]],
) -> dict[str, list[str]]:
    plain = _empty_filters()
    for key in VACANCY_FILTER_KEYS:
        plain[key] = [value for value, _ in filters[key]]
    return plain

def _apply_filters(
    stmt: Select[tuple[int]], filters: dict[str, list[str]]
) -> Select[tuple[int]]:
    for key, values in filters.items():
        if values:
            stmt =
stmt.where(_FILTER_COLUMN_MAP[key].in_(values))
    return stmt

def _normalize_role_keywords(payload: dict[str, Any]) ->
list[str]:
    role_keywords = payload.get("role_keywords", [])
    if not isinstance(role_keywords, list):
        return []
    normalized = sorted(
        {
            item.strip().lower()
            for item in role_keywords
            if isinstance(item, str) and item.strip()
        }
    )
    return normalized[:5]

def _apply_role_keywords(
    stmt: Select[tuple[int]], role_keywords: list[str]
) -> Select[tuple[int]]:
    if not ROLE_KEYWORDS_HARD_LOCK or not role_keywords:
        return stmt
    clauses = [
        or_(
            Vacancy.title.ilike(f"%{keyword}%"),
            Vacancy.skills.ilike(f"%{keyword}%"),
        )
        for keyword in role_keywords
    ]
    return stmt.where(or_(*clauses))

async def _count_filtered_candidates(
    filters: dict[str, list[str]],
    *,
    role_keywords: list[str],
) -> int:
    stmt = select(func.count(Vacancy.id))

```

```

stmt = _apply_filters(stmt, filters)
stmt = _apply_role_keywords(stmt, role_keywords)
async with async_session_factory() as session:
    count = await session.scalar(stmt)
return int(count or 0)

def _build_selected_value_counts(
    filters: dict[str, list[tuple[str, float]]],
    value_counts: dict[str, dict[str, int]],
) -> dict[str, dict[str, int]]:
    selected_counts = _empty_filter_counts()
    for key in VACANCY_FILTER_KEYS:
        for value, _ in filters.get(key, []):
            selected_counts[key][value] = value_counts.get(key,
{}).get(value, 0)
    return selected_counts

def _build_field_drop_order(
    weighted_filters: dict[str, list[tuple[str, float]]],
    protected_keys: set[str],
) -> list[tuple[str, float]]:
    fields: list[tuple[str, float]] = []
    for key in VACANCY_FILTER_KEYS:
        values = weighted_filters[key]
        if values:
            field_weight = max(weight for _, weight in values)
            fields.append((key, field_weight))

    def _effective_weight(item: tuple[str, float]) -> float:
        key, weight = item
        if key in protected_keys:
            return min(1.0, weight +
PROTECTED_RELAX_WEIGHT_PENALTY)
        return weight

    return sorted(
        fields,
        key=lambda item: (
            _effective_weight(item),
            item[0],
        ),
    )

async def _relax_filters(
    weighted_filters: dict[str, list[tuple[str, float]]],
    *,
    value_counts: dict[str, dict[str, int]],
    role_keywords: list[str],
    min_candidates: int,
) -> tuple[dict[str, list[str]], dict[str, list[str]],
list[FilterRelaxStep], int]:
    active_filters =
_clone_non_empty(_plain_filters_from_weighted(weighted_filters))
    weighted_by_key = {key: list(values) for key, values in
weighted_filters.items()}
    dropped_filters = _empty_filters()
    relax_steps: list[FilterRelaxStep] = []

```

```

candidate_count = await _count_filtered_candidates(
    active_filters, role_keywords=role_keywords
)
if candidate_count >= min_candidates:
    return (
        _with_all_keys(active_filters),
        dropped_filters,
        relax_steps,
        candidate_count,
    )

protected_keys = set(PROTECTED_RELAX_FILTER_KEYS)
drop_order = _build_field_drop_order(weighted_by_key,
protected_keys)
for step_index, (key, weight) in enumerate(drop_order,
start=1):
    removed_values = [value for value, _ in
weighted_by_key.get(key, [])]
    if not removed_values:
        continue
    candidates_before = candidate_count
    dropped_filters[key].extend(removed_values)
    active_filters.pop(key, None)
    candidate_count = await _count_filtered_candidates(
        active_filters, role_keywords=role_keywords
    )
    removed_total_count = sum(
        value_counts.get(key, {}).get(value, 0) for value in
removed_values
    )
    relax_steps.append(
        FilterRelaxStep(
            step=step_index,
            filter_key=key,
            filter_value=", ".join(removed_values),
            removed_weight=weight,
            value_count_in_db=removed_total_count,
            candidates_before=candidates_before,
            candidates_after=candidate_count,
        )
    )
    if candidate_count >= min_candidates:
        break
    return _with_all_keys(active_filters), dropped_filters,
relax_steps, candidate_count

def _query_domain_keywords(query: str, role_keywords: list[str])
-> set[str]:
    query_lower = query.lower()
    matched: set[str] = set(role_keywords)
    for keywords in DOMAIN_KEYWORDS.values():
        if any(keyword in query_lower for keyword in keywords):
            matched.update(keywords)
    return matched

def _rank_with_domain_boost(
    user_query: str,
    role_keywords: list[str],

```

```

        vacancies: list[SimilaritySearchResult],
    ) -> list[SimilaritySearchResult]:
        keywords = _query_domain_keywords(user_query, role_keywords)
        if not keywords:
            return vacancies
        scored: list[tuple[float, float, SimilaritySearchResult]] =
[]
        for vacancy in vacancies:
            haystack = f"{vacancy.title} {vacancy.skills}".lower()
            matches = sum(1 for keyword in keywords if keyword in
haystack)
            boosted_distance = vacancy.cosine_distance -
DOMAIN_BOOST_STEP * min(matches, 3)
            scored.append((boosted_distance,
vacancy.cosine_distance, vacancy))
            scored.sort(key=lambda item: (item[0], item[1]))
            return [item[2] for item in scored]

async def user_search(
    user_query: str,
    *,
    limit: int = DEFAULT_SIMILARITY_TOP_K,
    min_candidates: int = DEFAULT_MIN_FILTERED_CANDIDATES,
) -> UserSearchResult:
    """
    Run user-facing search flow:
    1) infer structured filters via LLM,
    2) relax low-weight fields until enough candidates,
    3) run embedding similarity search within filtered vacancy
subset.
    """
    stripped_query = user_query.strip()
    if not stripped_query:
        return UserSearchResult(
            user_query="",
            role_keywords=[],
            llm_filters=_empty_weighted_filters(),
            all_value_counts=_empty_filter_counts(),
            selected_value_counts=_empty_filter_counts(),
            applied_filters=_empty_filters(),
            dropped_filters=_empty_filters(),
            relax_steps=[],
            candidate_count=0,
            vacancies=[],
        )
    raw_filters = await
get_vacancy_filters_from_text_async(stripped_query)
    role_keywords = _normalize_role_keywords(raw_filters)
    llm_weighted_filters =
_normalize_weighted_filters(raw_filters)
    value_counts = await build_vacancy_filter_value_counts()
    selected_value_counts = _build_selected_value_counts(
        llm_weighted_filters, value_counts
    )
    (
        applied_filters,
        dropped_filters,
        relax_steps,
        candidate_count,

```

```

    ) = await _relax_filters(
        llm_weighted_filters,
        value_counts=value_counts,
        role_keywords=role_keywords,
        min_candidates=min_candidates,
    )
    vacancies = await similarity_search(
        stripped_query,
        limit=limit,
        filters=applied_filters,
        role_keywords=role_keywords,
    )
    ranked_vacancies = _rank_with_domain_boost(stripped_query,
        role_keywords, vacancies)
    return UserSearchResult(
        user_query=stripped_query,
        role_keywords=role_keywords,

llm_filters=_serialize_weighted_filters(llm_weighted_filters),
    all_value_counts=_with_all_count_keys(value_counts),
    selected_value_counts=selected_value_counts,
    applied_filters=applied_filters,
    dropped_filters=dropped_filters,
    relax_steps=relax_steps,
    candidate_count=candidate_count,
    vacancies=ranked_vacancies,
    )

def run_user_search(
    user_query: str,
    *,
    limit: int = DEFAULT_SIMILARITY_TOP_K,
    min_candidates: int = DEFAULT_MIN_FILTERED_CANDIDATES,
) -> UserSearchResult:
    """Synchronous wrapper for :func:`user_search` (CLI scripts,
    REPL)."""
    return asyncio.run(
        user_search(user_query, limit=limit,
        min_candidates=min_candidates)
    )

```

Листинг A.5 – Файл backend/src/features/search/service.py

Примечание – Источник: собственная разработка.

```

"""Compute and persist vacancy embeddings (weighted structured
fields; description optional)."""

from __future__ import annotations

import asyncio
from typing import Any

from sqlalchemy import select, update
from sqlalchemy.ext.asyncio import AsyncSession

```

```

from common.constants.embedding import (
    DEFAULT_EMBED_BATCH_SIZE,
    VACANCY_EMBED_COMPANY_REPEATS,
    VACANCY_EMBED_DESCRIPTION_MAX_CHARS,
    VACANCY_EMBED_INCLUDE_DESCRIPTION,
    VACANCY_EMBED_SKILLS_REPEATS,
    VACANCY_EMBED_STRUCTURE_REPEATS,
    VACANCY_EMBED_TITLE_REPEATS,
)
from features.database.db import async_session_factory
from features.database.models import Vacancy
from features.embedding.encoder import encode_texts,
require_pgvector_embedding

_SKIP_PLACEHOLDERS: frozenset[str] = frozenset(
    {"", "не указано", "не указаны", "-"},
)

_STRUCTURE_FIELDS: tuple[tuple[str, str], ...] = (
    ("Занятость", "employment"),
    ("График", "schedule"),
    ("Часы", "hours"),
    ("Формат работы", "work_format"),
    ("Опыт", "experience"),
    ("Трудоустройство", "hiring_format"),
)

def _trim(s: str) -> str:
    return s.strip()

def _skip_value(value: str) -> bool:
    t = _trim(value).lower()
    return not t or t in _SKIP_PLACEHOLDERS

def _repeat_lines(label: str, value: str, times: int) -> list[str]:
    if _skip_value(value):
        return []
    line = f"{label}: {_trim(value)}"
    return [line] * times

def _description_for_embed(raw: str) -> str | None:
    """Return description slice for embedding, or None if
    disabled / empty."""
    if not VACANCY_EMBED_INCLUDE_DESCRIPTION:
        return None
    t = _trim(raw)
    if not t:
        return None
    cap = VACANCY_EMBED_DESCRIPTION_MAX_CHARS
    if cap is not None and len(t) > cap:
        return t[:cap].rstrip() + "..."
    return t

def _vacancy_embed_text(row: Any) -> str:

```

```

"""
    Build one string for embedding: title and structured fields
    (weighted by repetition),
    then skills. Description is optional (see
    ``VACANCY_EMBED_INCLUDE_DESCRIPTION``).
"""
    chunks: list[str] = []

    title = _trim(row.title)
    if title:
        job_line = f"Должность: {title}"
        chunks.extend([job_line] * VACANCY_EMBED_TITLE_REPEATS)

    chunks.extend(_repeat_lines("Компания", row.company,
                                VACANCY_EMBED_COMPANY_REPEATS))
    for label, attr in _STRUCTURE_FIELDS:
        chunks.extend(
            _repeat_lines(
                label,
                getattr(row, attr),
                VACANCY_EMBED_STRUCTURE_REPEATS,
            )
        )
    chunks.extend(_repeat_lines("Зарплата", row.salary, 1))
    chunks.extend(_repeat_lines("Выплаты",
                                row.payment_frequency, 1))

    if not _skip_value(row.skills):
        sk = f"Навыки: {_trim(row.skills)}"
        chunks.extend([sk] * VACANCY_EMBED_SKILLS_REPEATS)

    desc = _description_for_embed(row.description)
    if desc:
        chunks.append(f"Описание: {desc}")

    return "\n\n".join(chunks)

async def _fetch_batch(
    session: AsyncSession,
    *,
    after_id: int,
    batch_size: int,
    force: bool,
) -> list[Any]:
    stmt = (
        select(
            Vacancy.id,
            Vacancy.title,
            Vacancy.company,
            Vacancy.salary,
            Vacancy.payment_frequency,
            Vacancy.experience,
            Vacancy.employment,
            Vacancy.hiring_format,
            Vacancy.schedule,
            Vacancy.hours,
            Vacancy.work_format,
            Vacancy.skills,
            Vacancy.description,

```

```

        )
        .where(Vacancy.id > after_id)
        .order_by(Vacancy.id)
        .limit(batch_size)
    )
    if not force:
        stmt = stmt.where(Vacancy.embedding.is_(None))
    result = await session.execute(stmt)
    return list(result.all())

async def _persist_vectors(
    session: AsyncSession,
    ids: list[int],
    vectors: list[list[float]],
) -> None:
    for vid, vec in zip(ids, vectors, strict=True):
        await session.execute(
            update(Vacancy).where(Vacancy.id ==
vid).values(embedding=vec)
        )
    await session.commit()

async def _embed_and_store_batch(
    session: AsyncSession,
    rows: list[Any],
) -> int:
    texts = [_vacancy_embed_text(r) for r in rows]
    vectors = await encode_texts(texts)
    ids = [r.id for r in rows]
    await _persist_vectors(session, ids, vectors)
    return len(ids)

async def fill_db_with_vectors(
    *,
    batch_size: int = DEFAULT_EMBED_BATCH_SIZE,
    force: bool = False,
) -> int:
    """
    Fill ``vacancies.embedding`` using Sentence-BERT
    (multilingual MiniLM, dim 384).

    Text combines weighted structured fields (title, schedule,
    employment, ...), skills,
    and optionally the full description (see
    ``VACANCY_EMBED_INCLUDE_DESCRIPTION``). After
    changing embedding settings in
    ``common.constants.embedding``, run with ``force=True``
    to refresh all vectors.

    By default updates only rows with ``embedding IS NULL``. Set
    ``force=True`` to
    recompute embeddings for every vacancy (keyset pagination by
    ``id``).

    Returns the number of rows updated.
    """
    require_pgvector_embedding()

```



```

total = 0
after_id = 0
async with async_session_factory() as session:
    while True:
        rows = await _fetch_batch(
            session,
            after_id=after_id,
            batch_size=batch_size,
            force=force,
        )
        if not rows:
            break
        after_id = rows[-1].id
        total += await _embed_and_store_batch(session, rows)
    return total

def run_fill_db_with_vectors(
    *,
    batch_size: int = DEFAULT_EMBED_BATCH_SIZE,
    force: bool = False,
) -> int:
    """
    Run :func:`fill_db_with_vectors` from synchronous code (CLI,
    scripts).

    Wraps ``asyncio.run`` so callers do not need to manage an
    event loop.
    """
    return
    asyncio.run(fill_db_with_vectors(batch_size=batch_size,
    force=force))

```

Листинг А.6 – Файл backend/src/features/embedding/save_vectors_service.py

Примечание – Источник: собственная разработка.

```

"""Streamlit chat UI for AI Job Finder frontend."""

from __future__ import annotations

import streamlit as st

from constants.messages import (
    ASSISTANT_RUNNING_MESSAGE,
    CHAT_INPUT_PLACEHOLDER,
    INITIAL_ASSISTANT_MESSAGE,
)
from constants.ui import (
    CHAT_ASSISTANT_NAME,
    CHAT_TITLE,
    PAGE_ICON,
    PAGE_LAYOUT,
    SESSION_MESSAGES_KEY,
    SESSION_VACANCY_EXPANDED_KEY,
)
from dto.chat_dto import ChatMessageDto

```

```

from services.chat_service import ChatService
from ui.vacancy_view import render_vacancy_card

def _init_state() -> None:
    """Initialize Streamlit session state for chat history."""
    if SESSION_MESSAGES_KEY not in st.session_state:
        st.session_state[SESSION_MESSAGES_KEY] = [
            ChatMessageDto(
                role="assistant",
                content=INITIAL_ASSISTANT_MESSAGE,
            )
        ]
    if SESSION_VACANCY_EXPANDED_KEY not in st.session_state:
        st.session_state[SESSION_VACANCY_EXPANDED_KEY] = {}

def _render_messages() -> None:
    """Render existing chat messages."""
    for index, message in
    enumerate(st.session_state[SESSION_MESSAGES_KEY]):
        with st.chat_message(message.role):
            st.markdown(message.content)
            vacancies = message.vacancies
            if vacancies is None and message.vacancy is not
None:
                vacancies = [message.vacancy]
            if not vacancies:
                continue
            for vacancy_index, vacancy in enumerate(vacancies):
                render_vacancy_card(
                    vacancy,
                    message_index=index,
                    vacancy_index=vacancy_index,
                )

def _append_user_message(prompt: str) -> None:
    """Append one user message to history."""
    st.session_state[SESSION_MESSAGES_KEY].append(
        ChatMessageDto(role="user", content=prompt)
    )

def _append_assistant_response(prompt: str) -> None:
    """Generate and append assistant response with vacancy."""
    chat_service = ChatService()
    response = chat_service.build_response(prompt)
    st.session_state[SESSION_MESSAGES_KEY].append(response)

def main() -> None:
    """Run Streamlit application entrypoint."""
    st.set_page_config(page_title=CHAT_TITLE,
        page_icon=PAGE_ICON, layout=PAGE_LAYOUT)
    st.title(CHAT_TITLE)
    st.caption(f"Чат-ассистент: {CHAT_ASSISTANT_NAME}")

    _init_state()
    _render_messages()

```

```

prompt = st.chat_input(CHAT_INPUT_PLACEHOLDER)
if not prompt:
    return

_append_user_message(prompt)
with st.chat_message("assistant"):
    with st.spinner(ASSISTANT_RUNNING_MESSAGE):
        _append_assistant_response(prompt)
st.rerun()

if __name__ == "__main__":
    main()

```

Листинг А.7 – Файл frontend/src/app.py

Примечание – Источник: собственная разработка.

```

"""Chat orchestration service for Streamlit app."""

from __future__ import annotations

from dto.chat_dto import ChatMessageDto
from dto.vacancy_dto import VacancyDto
from services.backend_api_service import BackendApiService

class ChatService:
    """Build assistant messages from user profile prompts."""

    def __init__(self) -> None:
        self.backend_service = BackendApiService()

    @staticmethod
    def _sort_vacancies_by_similarity(vacancies:
list[VacancyDto]) -> list[VacancyDto]:
        """Sort vacancies so the smallest cosine distance
appears first."""
        return sorted(
            vacancies,
            key=lambda vacancy: (
                vacancy.cosine_distance is None,
                vacancy.cosine_distance if
vacancy.cosine_distance is not None else float("inf"),
            ),
        )

    def build_response(self, user_prompt: str) ->
ChatMessageDto:
        """Create one assistant message with 0..5 vacancy
cards."""
        analyzed =
self.backend_service.get_vacancy_for_profile(user_prompt)
        if not analyzed.vacancies:
            return ChatMessageDto(role="assistant",
content=analyzed.summary)

```

```

        sorted_vacancies =
self._sort_vacancies_by_similarity(analyzed.vacancies)
        count = len(sorted_vacancies)
        text = f"{analyzed.summary}\n\nНайдено вакансий:
**{count}**."
        return ChatMessageDto(
            role="assistant", content=text,
vacancies=sorted_vacancies
        )

```

Листинг A.8 – Файл frontend/src/services/chat_service.py

Примечание – Источник: собственная разработка.

ПРИЛОЖЕНИЕ Б

Набор бенчмарк-запросов и результаты тестирования

В приложении приведён полный перечень эталонных запросов, использованных для количественной оценки качества пользовательского поиска вакансий, и результаты их тестирования на разработанном чат-боте. Набор сформирован по материалам модуля backend/src/features/search/evaluation.py (функция run_search_evaluation) и включает – в соответствии с методикой, описанной в параграфе 4.1 основного текста, шестнадцать естественно-языковых запросов, охватывающих типовые сценарии поиска.

Контрольное тестирование выполнено в локальной среде проекта на актуальной базе вакансий портала rabota.by. Между обращениями к языковой модели Groq выдерживалась задержка в шестьдесят секунд для соблюдения ограничения частоты обращений к Chat Completions API. Для каждого запроса в таблице Б.1 указан сокращённый текст обращения, кортеж ожидаемых ключевых слов, значения метрик hit@1 и hit@5, а также заголовок вакансии, занявшей первую позицию выдачи. Полный машинный протокол тестирования с косинусными расстояниями и идентификаторами вакансий хранится в файле evaluation-test-2.txt репозитория проекта.

Таблица Б.1 – Результаты тестирования эталонных запросов

№	Запрос (сокращённо)	Ожидаемые ключевые слова	hit@1	hit@5	Заголовок вакансии в позиции top-1
1	Студент без опыта, ПВЗ Wildberries или Ozon, на месте работодателя	пвз, пункт выдачи, wildberries, ozon	да	да	Продавец-кассир в пункт выдачи заказов, город Гродно
2	Стажировка или junior в интернет-маркетинге, digital	маркетолог, интернет-маркетолога, digital	да	да	Ассистент интернет-маркетолога

3	Бариста или работа в кофейне, без опыта	бариста, кофе, кофейн	да	да	Бариста (ИП Христинченко Татьяна Петровна)
4	Кладовщик 2 на 2, вечерние или ночные смены, на месте работодателя	кладовщик, склад	да	да	Кладовщик (ООО ЛогиЛидер)
5	Комплектовщик или сортировщик на складе, ночные смены	комплектовщик, сортировщик, склад	да	да	Комплектовщик / Грузчик / Разнорабочий
6	Логист, экспедитор или менеджер по логистике, график 5 на 2	логист, экспедитор	да	да	Менеджер по логистике (ООО К 2 Технологии)
7	Подработка продавцом в магазине, частичная занятость	продавец	да	да	Продавец в магазин «Гренка»
8	Удалённая работа support или help desk specialist	support, help desk, специалист	да	да	Support Specialist (Help Desk Analyst), GP Solutions
9	Программист 1С, удалённо, полная занятость	программист 1с, 1с	да	да	Программист 1С (ОДО ЮКОЛА-ИНФО-Брест)

10	Официант или повар без опыта, сменный график	официант, повар	да	да	Официант в ресторан «МАРМО»
11	Водитель категории В, разъездной формат и доставка	водитель	да	да	Водитель категории В (Автолайтэкспресс)
12	Стартовая роль в подборе персонала, IT-сфера, стажировка или junior	подбор, hr, рекрутер, персонал	нет	да	Junior Sales Manager (ООО Сенлайн)
13	Удалённый ассистент интернет-маркетолога, контент и аналитика	ассистент интернет-маркетолога, маркетолог	да	да	Ассистент интернет-маркетолога (Зизор)
14	Мерчендайзер или торговый представитель, разъездной характер	мерчендайзер, торгов	да	да	Торговый представитель отдела продаж
15	Кондитер или помощник кондитера, сменный график, десерты	кондитер, десерт	да	да	Кондитер / помощник кондитера (СвитБэйкер, ЧП)

16	ПВЗ Ozon или Wildberries, выдача посылок, приём заказов	пвз, ozon, wildberries, пункт выдачи	нет	нет	Подходящих вакансий не найдено
----	---	--------------------------------------	-----	-----	--------------------------------

Окончание таблицы Б.1

Примечание – Источник: собственная разработка.

Итоговые значения метрик по всему набору бенчмарков составили: hit@1 равен четырнадцати из шестнадцати, hit@5 равен пятнадцати из шестнадцати. Доля корректных попаданий целевой профессиональной роли в первую позицию выдачи составила приблизительно ноль целых восемьдесят восемь сотых, доля корректных попаданий в топ-5 – приблизительно ноль целых девяносто четыре сотых. Полученные значения превышают целевое нефункциональное требование hit@5 не ниже ноль целых семьдесят пять сотых, зафиксированное в параграфе 2.3 основного текста, что подтверждает работоспособность чат-бота на эталонных пользовательских сценариях.

Из шестнадцати запросов лишь два не дали идеального попадания. В сценарии под номером двенадцать (стартовая роль в подборе персонала, IT-сфера, формат стажировки или junior) релевантная вакансия присутствовала в топ-5, однако первой позицией выдачи оказалась семантически близкая, но нерелевантная вакансия Junior Sales Manager. В сценарии под номером шестнадцать (работа в пункте выдачи заказов Ozon или Wildberries с упором на выдачу посылок) после применения LLM-фильтров и нескольких этапов релаксации в локальной базе не осталось ни одного кандидата, удовлетворяющего ограничениям; в этом случае серверная часть корректно вернула пустой список вакансий, что соответствует предусмотренной модели мягкой деградации, описанной в параграфе 4.4.

Сравнение полученных значений с показателями ранее выполненной версии прототипа (десять из шестнадцати по hit@1 и тринадцать из шестнадцати по hit@5) показывает заметное улучшение по обоим метрикам. Прирост достигнут за счёт реализации алгоритма доменного ранжирования и адаптивной релаксации LLM-фильтров с защитой приоритетных ключей (параграф 3.5), а также взвешенного текстового представления вакансии при векторизации (параграф 3.7). Таким образом, представленный в приложении

Б протокол подтверждает эффективность принятых архитектурных и алгоритмических решений.